HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

Mobile Programming

(IT4785/IT4785E)



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Lesson 6: Display lists and use Material Design (part 2)

ONE LOVE. ONE FUTURE.

Recap: Goals

Kotlin fundamentals

- Functional programming
- Generics variance
- Collection internals
- Advanced design patterns

Composables

- Scrollable list
- App icon
- Grid
- Material design, theming
- Animation
- Accessibility
- Practice with real app: Superheroes

HUST

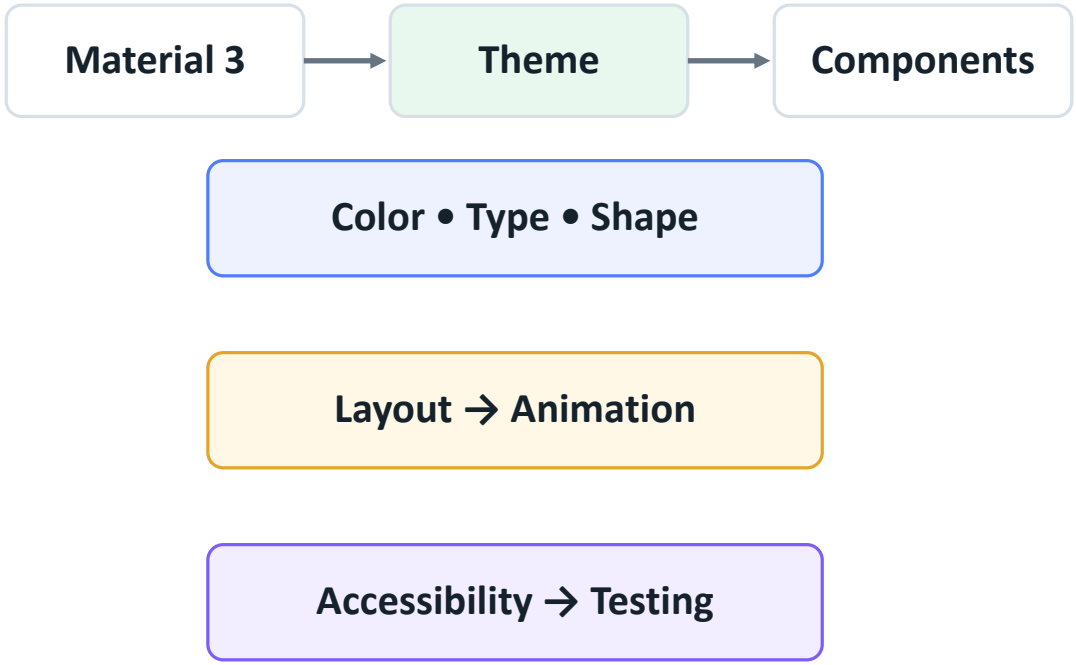
hust.edu.vn fb.com/dhbkhn

PATHWAY 3 - Build beautiful apps

- Material design
- Material theming
- Animation
- Accessibility
- Practice app: Superheroes

The Learning Journey

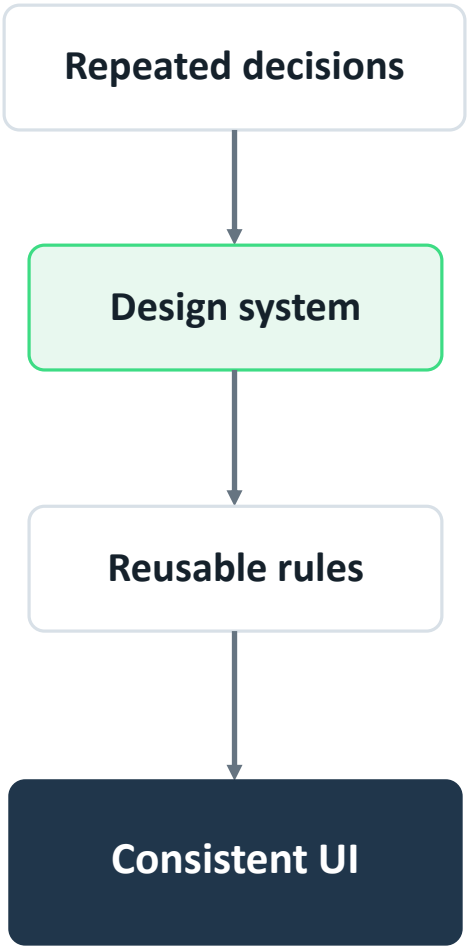
- Material Design → Material 3
- Theme → Color + Typography + Shape
- Components → Layout → Animation
- Accessibility → Testing → Production UI



From UI to Design System

The progression

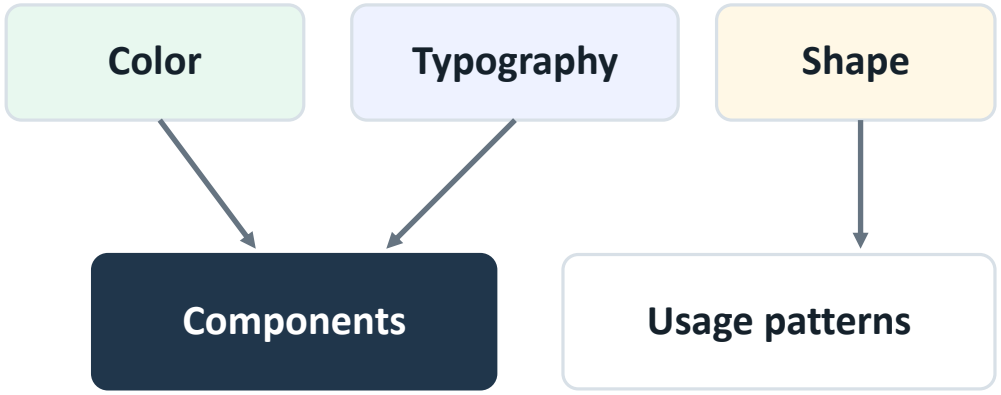
- Repeated design decisions
- Explicit rules and tokens
- Reusable components
- Consistent product experience



What Is a Design System?

A reusable system of

- Design rules
- Tokens / values
- Components
- Usage guidance
- Patterns

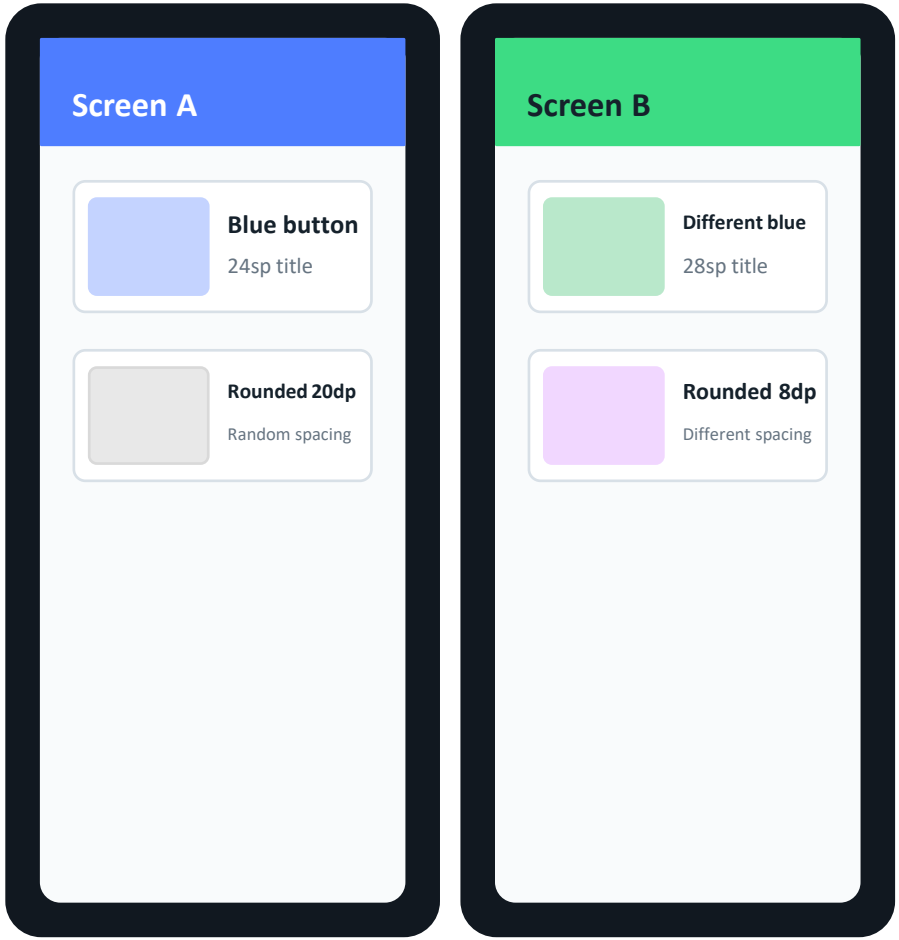


goal: consistency at scale

Why Design Systems Matter

Without a system

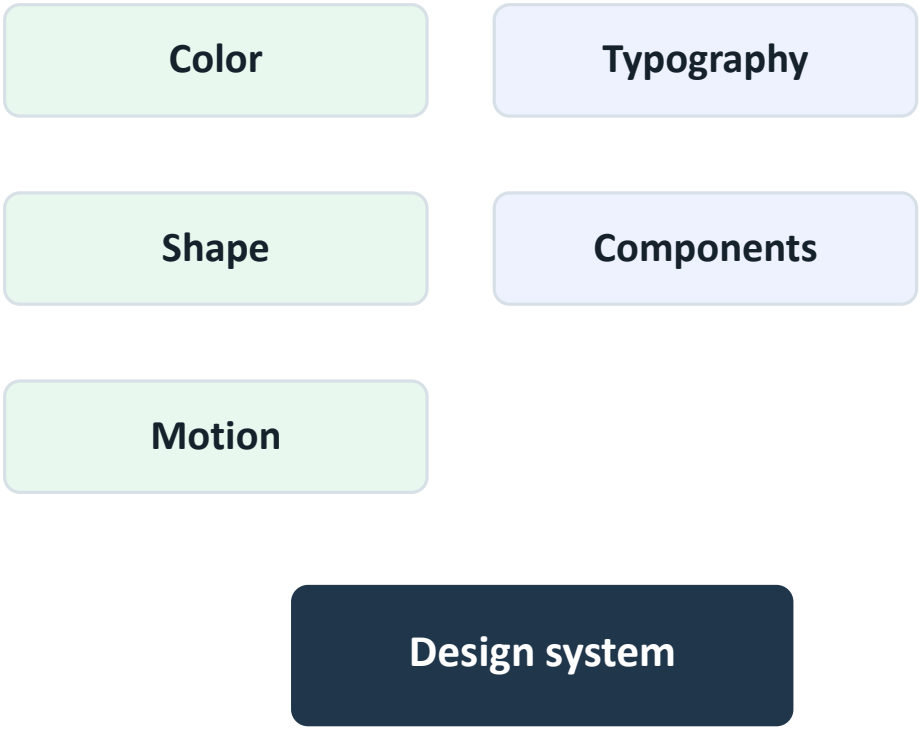
- Every screen invents a color
- Typography drifts
- Shapes become inconsistent
- Refactoring becomes expensive



Material Design

Material provides guidance for

- Color
- Typography
- Shape
- Components
- Layout
- Motion
- Accessibility



Material 2 vs Material 3

- Material 3 is the current Material design system
- Theme concepts use semantic roles
- Component APIs come from **androidx.compose.material3**
- Use M3 consistently

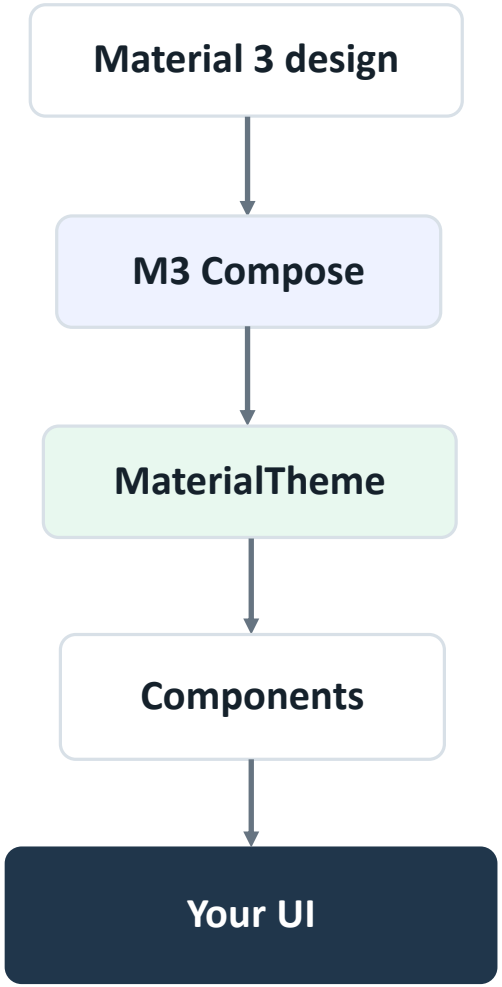
Material 2	Material 3
Earlier theme model	Semantic color roles

Current M3 → mention M2 only for awareness

Material 3 + Jetpack Compose

The implementation path

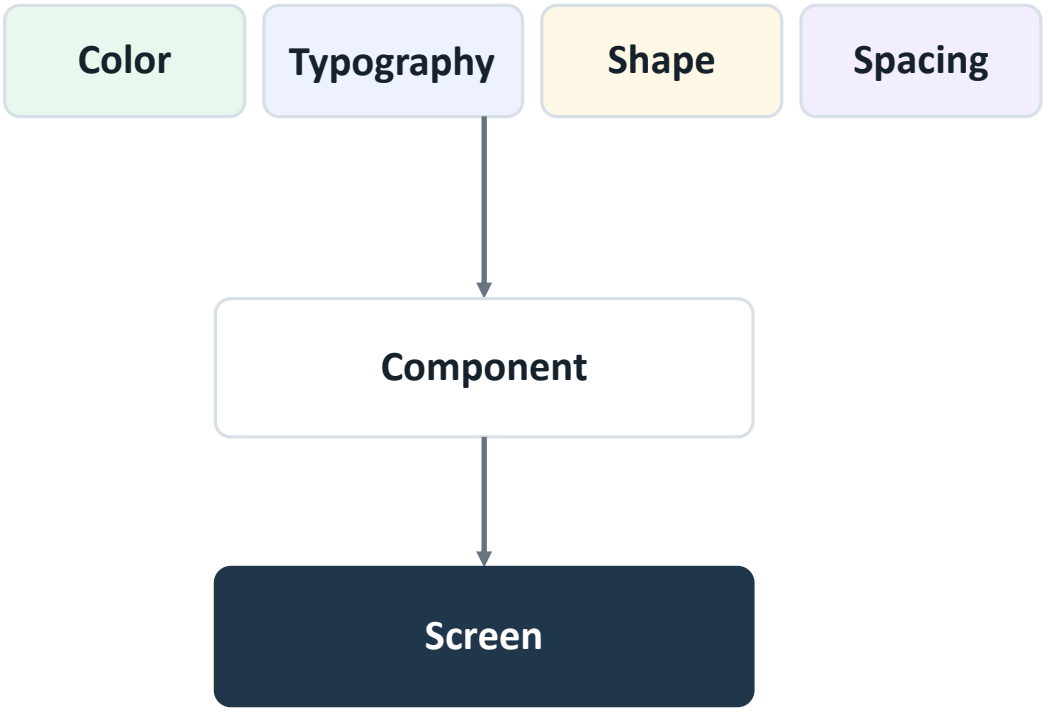
- Material 3 design
- Material 3 Compose library
- MaterialTheme
- Material components
- Application UI



Design System Tokens

Typical tokens

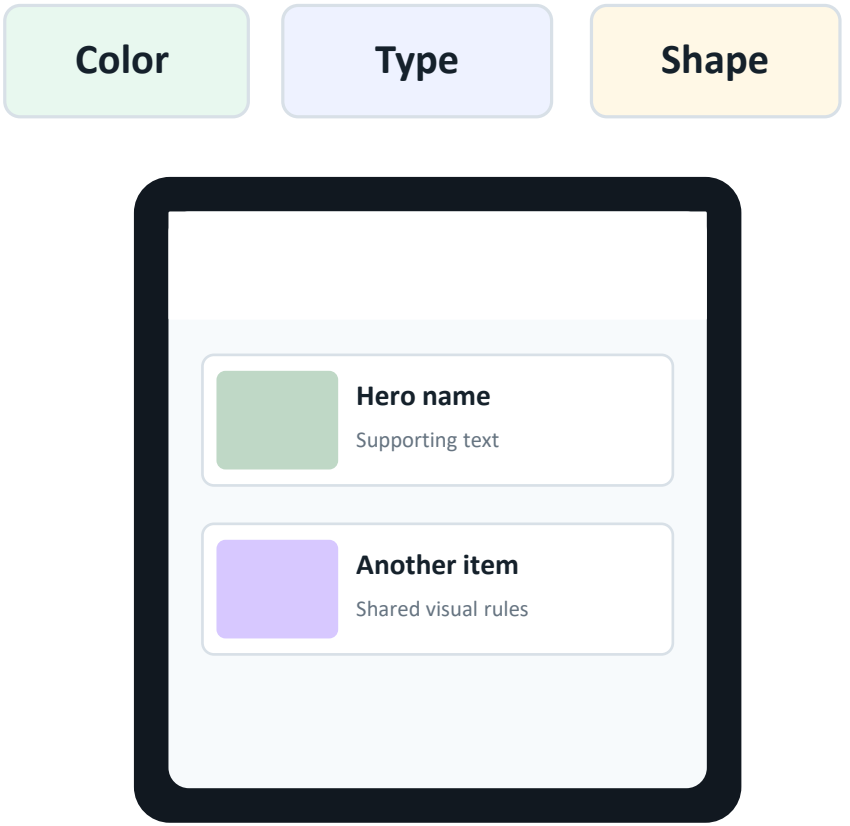
- Color roles
- Typography roles
- Shape roles
- Spacing
- Component behavior



From Tokens to Components

Example: one Card

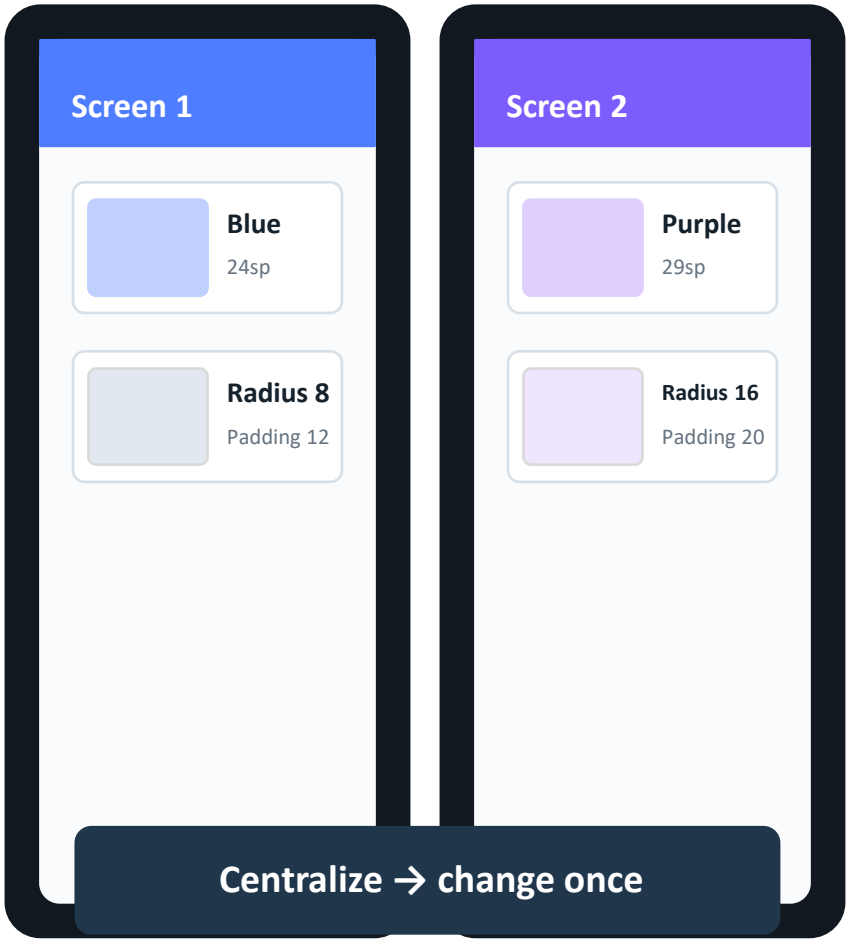
- Color role → container/content
- Typography role → title/body
- Shape role → corners
- Spacing rule → padding
- Component → reusable pattern



Why Theme Exists

The scaling problem

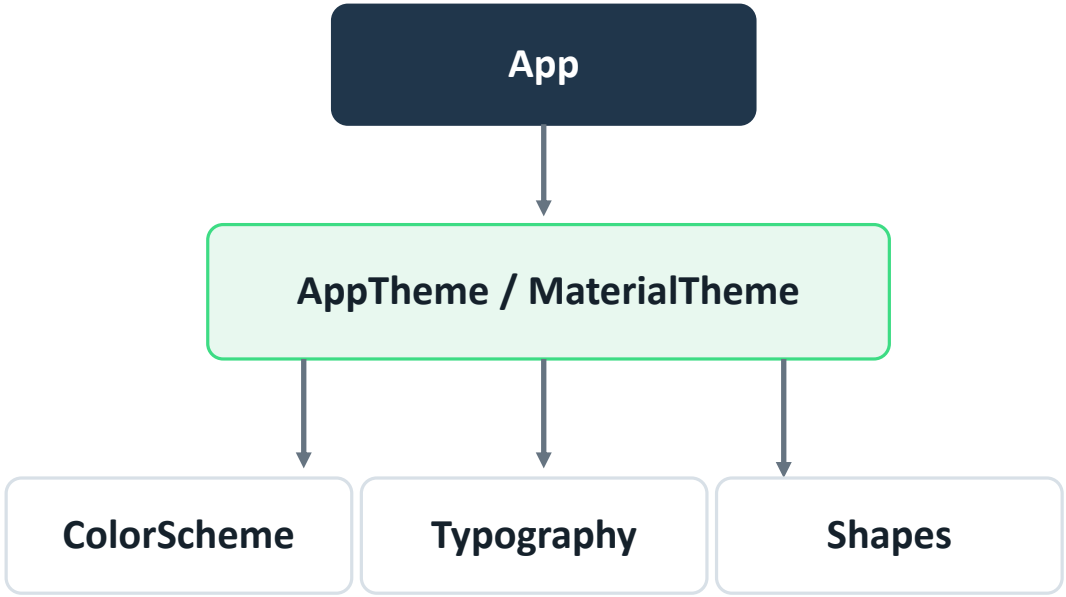
- 20 screens × multiple colors
- Typography choices drift
- Dark mode becomes expensive
- Updates require repeated edits



Theme Architecture

The basic structure

- AppTheme sits high in the composition tree
- MaterialTheme exposes theme values
- Child composables consume the values
- Components not need to know where the values came from



MaterialTheme

Core structure

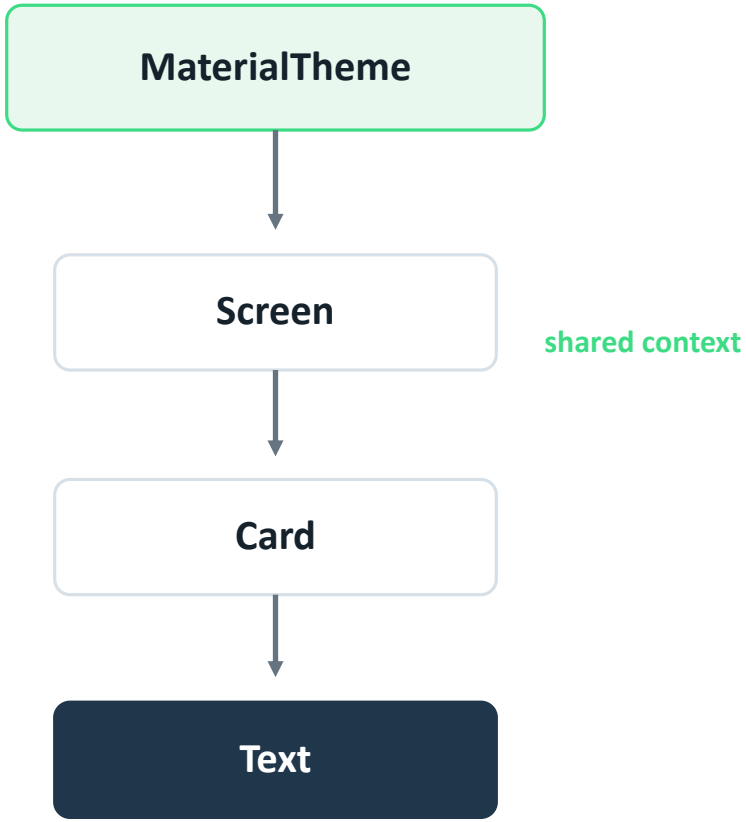
- Define values near the top
- Pass them to MaterialTheme
- Read them from descendants
- Keep component code focused on content and behavior

```
MaterialTheme(
    colorScheme = AppColors,
    typography = AppTypography,
    shapes = AppShapes
) {
    AppContent()
}
```

Theme Propagation

Read from anywhere below the theme

- Screen can access colorScheme
- Card can access typography
- Text can access shapes if needed
- No need to thread every design value manually



Reading the Theme

In Compose

- MaterialTheme.colorScheme
- MaterialTheme.typography
- MaterialTheme.shapes

```
MaterialTheme.colorScheme.primary

MaterialTheme.typography.headlineSmall

MaterialTheme.shapes.medium
```

Color | **Type** | **Shape**

Before vs After Theming

Refactor the same Text

- Before: literal color and size
- After: theme role + type role
- The content stays the same
- The design system becomes replaceable

```
// BEFORE
Text(
  text = "Hello",
  color = Color(0xFF123456),
  fontSize = 24.sp
)
```

```
// AFTER
Text(
  text = "Hello",
  color = MaterialTheme.colorScheme.primary,
  style = MaterialTheme.typography.headlineSmall
)
```

Why Hard-Coding Does Not Scale

Hard-coded screens become a patchwork

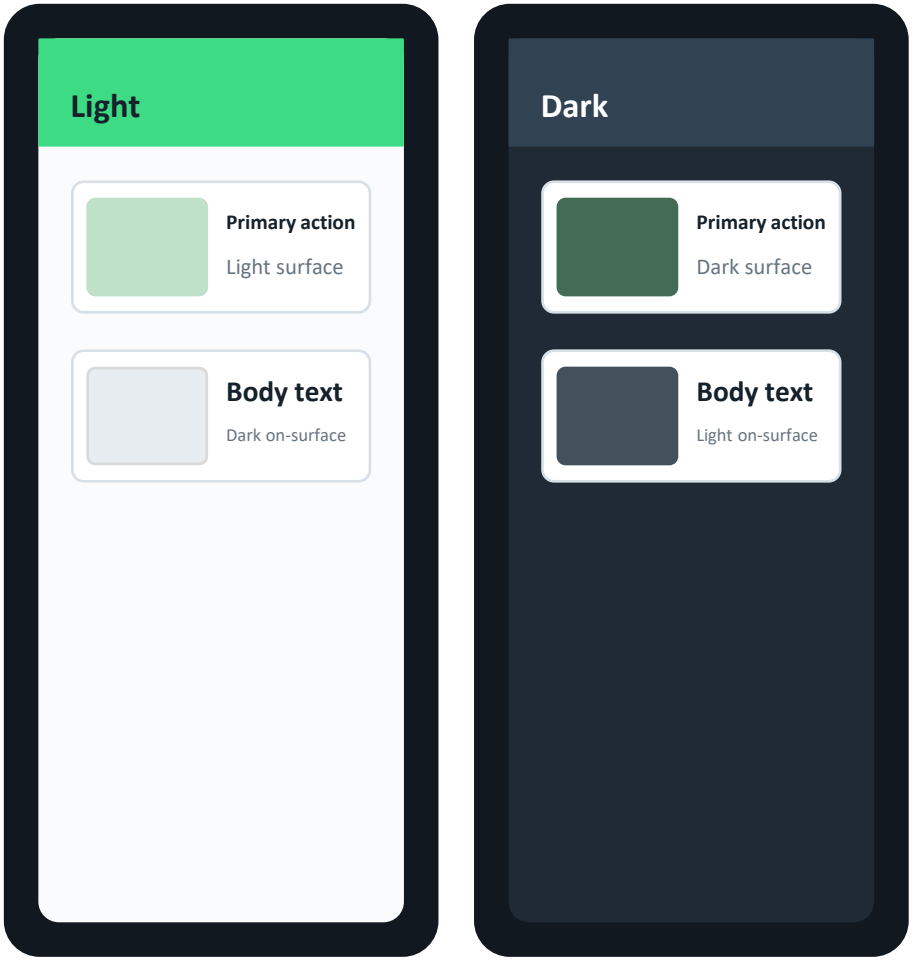
- Different blue values
- Different text sizes
- Different corner radii
- Manual dark-mode fixes



Light Theme vs Dark Theme

The component stays conceptually stable

- Light / dark scheme changes the visual palette
- Semantic roles stay the same
- Avoid per-component dark-mode conditionals



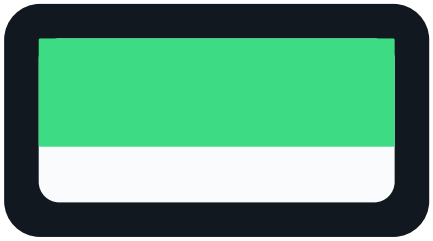
Theme Refactoring

```
// START
Text(
  text = "Welcome",
  color = Color.Blue,
  fontSize = 28.sp
)
```

```
// STEP 1
color = MaterialTheme.colorScheme.primary

// STEP 2
style = MaterialTheme.typography.headlineSmall
```

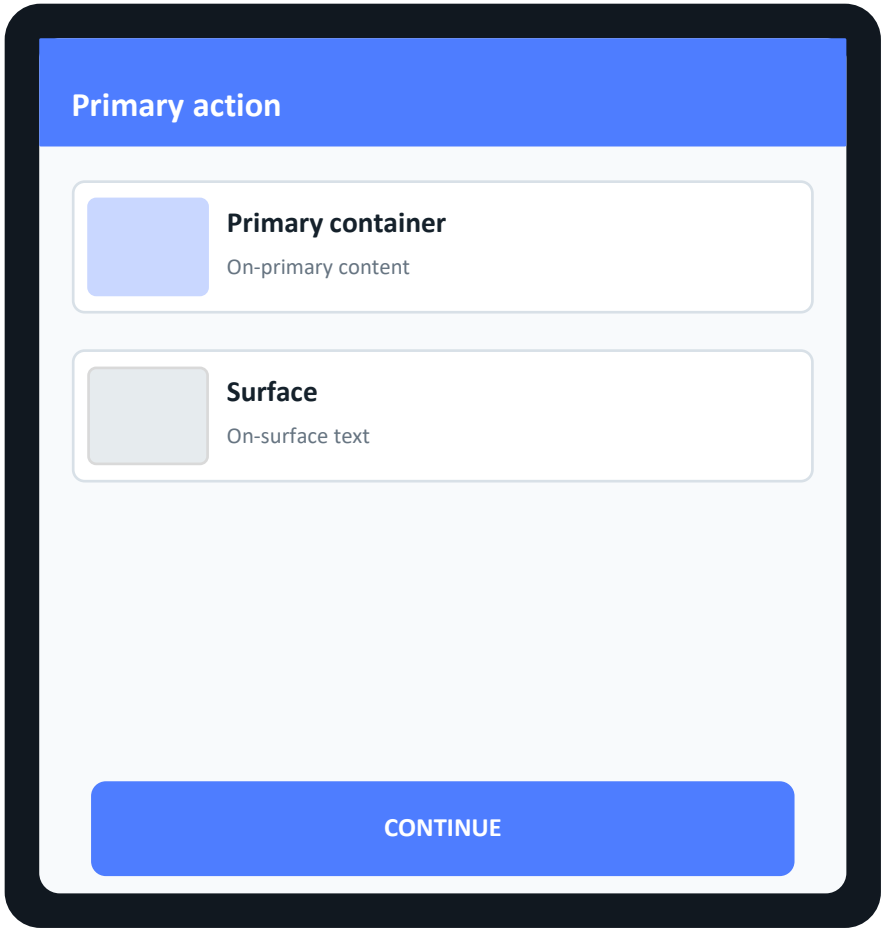
Run → switch theme → observe



Why Color Needs Roles

Semantic color asks…

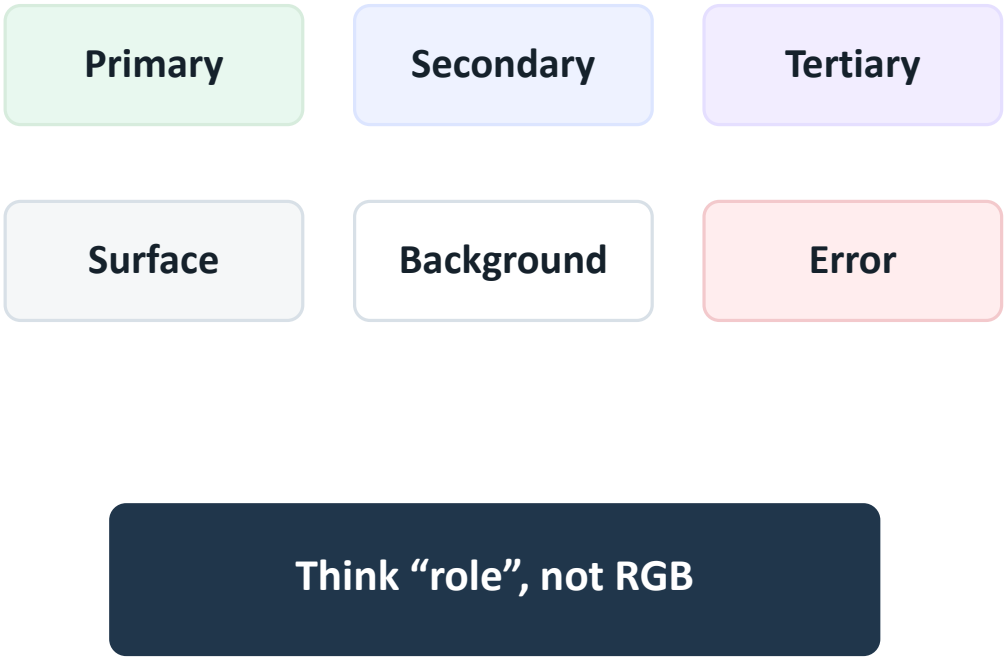
- What is this surface for?
- What content sits on it?
- How should it work in dark theme?
- Can users read it?



Material 3 Color Roles

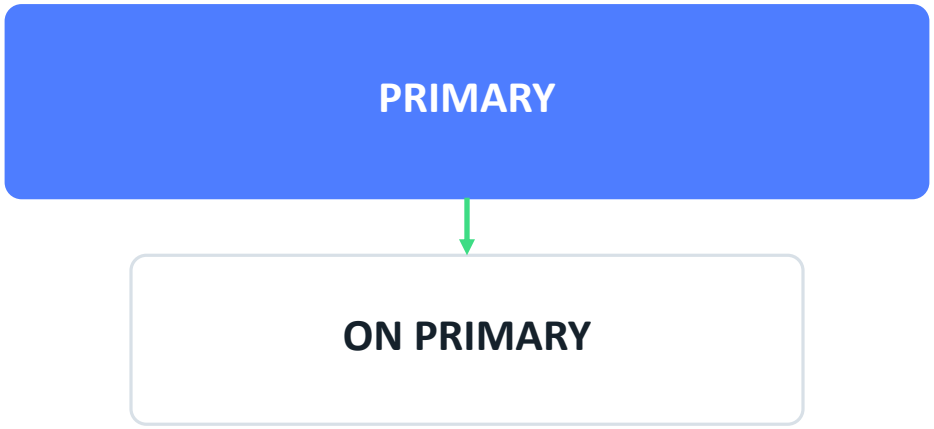
Core roles

- Primary
- Secondary
- Tertiary
- Surface
- Background
- Error



Paired Roles: Primary / On Primary

- Primary = container / surface role
- OnPrimary = content shown on it
- The same pattern appears elsewhere
- Semantic pairing supports readability



Primary vs On Primary in a Button

The semantic pair in code

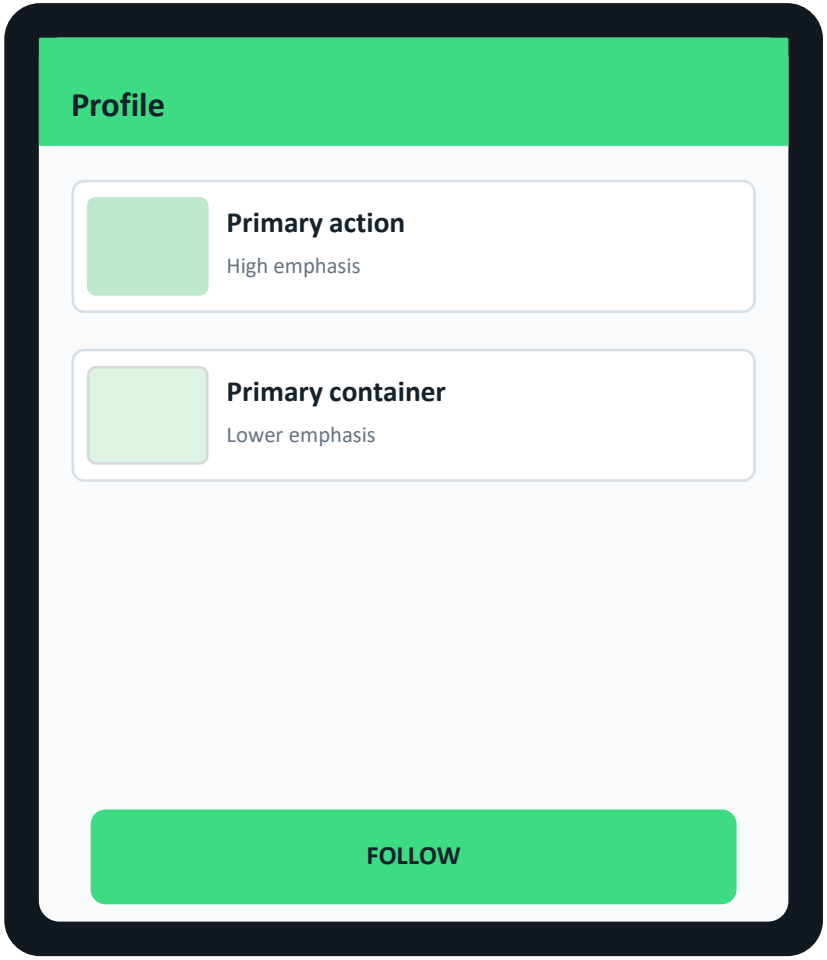
- containerColor → primary
- contentColor → onPrimary
- Do not invent a second text color
- Theme can change both together

```
Button(
    colors = ButtonDefaults.buttonColors(
        containerColor = MaterialTheme.colorScheme.primary,
        contentColor = MaterialTheme.colorScheme.onPrimary
    ),
    onClick = {}
) {
    Text("Continue")
}
```

Containers: Primary Container

Use cases

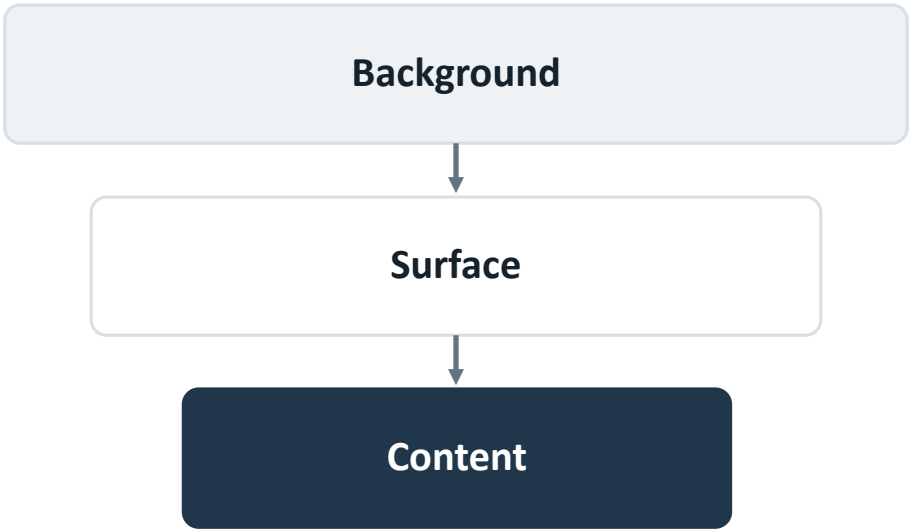
- Primary → strong emphasis / main action
- PrimaryContainer → supportive emphasis
- OnPrimaryContainer → readable content on it
- Keep the visual relationship coherent



Surface Hierarchy

Think in layers

- Background = overall screen
- Surface = container layer
- Content = readable foreground
- Use roles to preserve hierarchy in themes



Contrast Lab

Contrast ratio

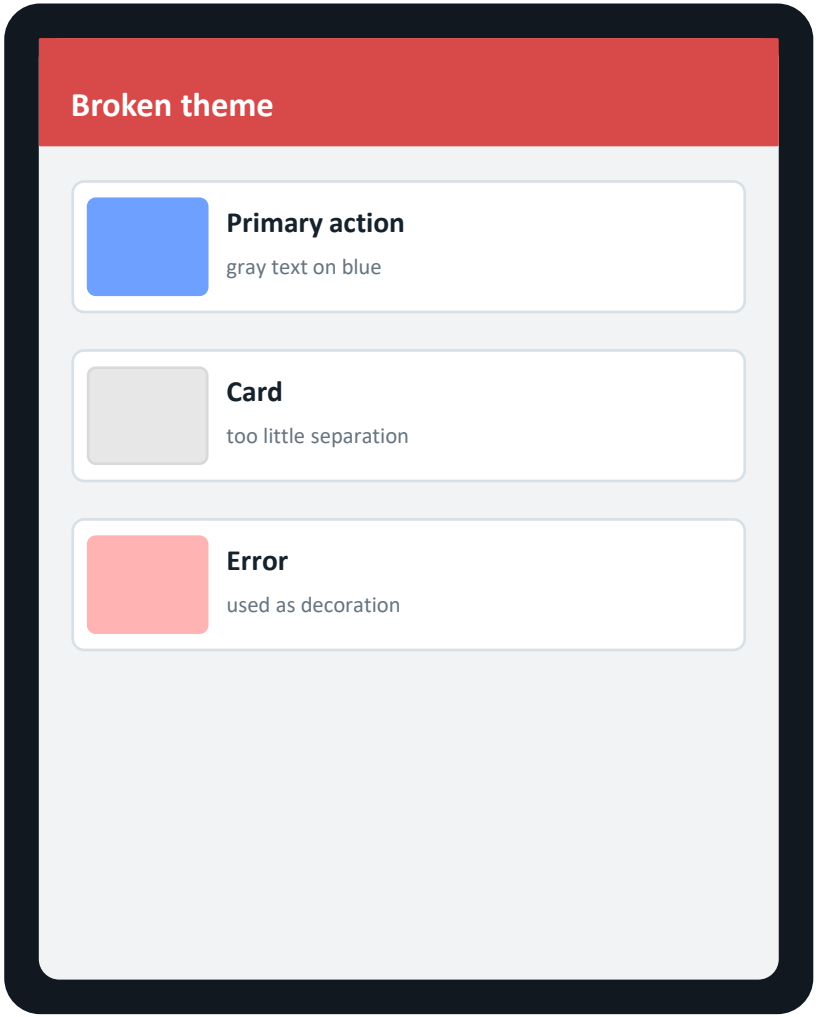
- Measures luminance difference
- Normal text: WCAG AA target 4.5:1
- Large text: 3:1
- AAA targets are stricter
- Check custom color combinations



Color Debugging Challenge

Find at least four problems

- Arbitrary blue instead of semantic role
- Low-contrast foreground
- Surface hierarchy is unclear
- Dark mode becomes unreadable
- Error color is reused for decoration



Typography Is Hierarchy

Good hierarchy helps users…

- Know what to read first
- Scan sections quickly
- Distinguish title vs supporting text
- Understand actions and metadata

Superheroes

Meet the heroes

Explore characters, stories, and strengths.

12 HEROES

Display / Headline / Body / Label

Material 3 Typography Scale

The hierarchy

- Display → very large emphasis
- Headline → major screen or section titles
- Title → component titles
- Body → readable content
- Label → compact supporting / action text

Display

Headline

Title

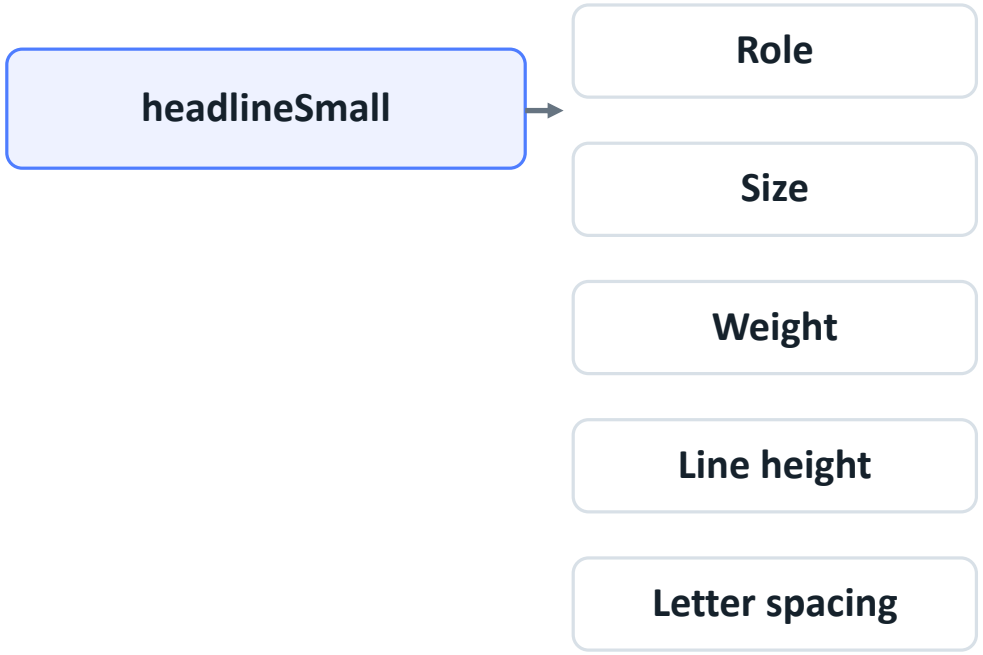
Body

Label

Typography Anatomy

A text style includes

- Role
- Font size
- Weight
- Line height
- Letter spacing



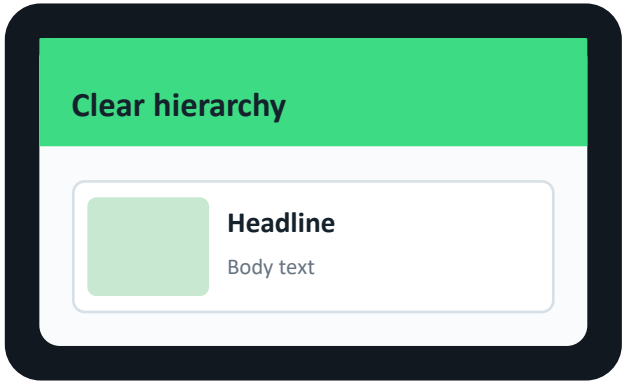
Typography Before → After

Refactoring principle

- Replace size literals
- Use MaterialTheme.typography
- Keep hierarchy consistent
- Review the result in context

```
// BEFORE
fontSize = 29.sp
fontSize = 17.sp
fontSize = 12.sp
```

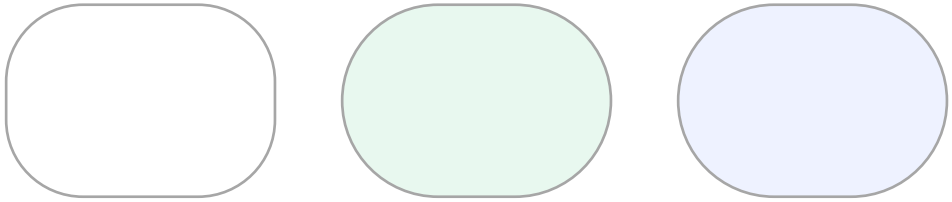
```
// AFTER
headlineSmall
bodyLarge
labelMedium
```



What Is Shape?

Shape influences

- Visual grouping
- Component identity
- Density and tone
- Consistency across screens



4dp 12dp 28dp

shape is a design rule

Shape Scale

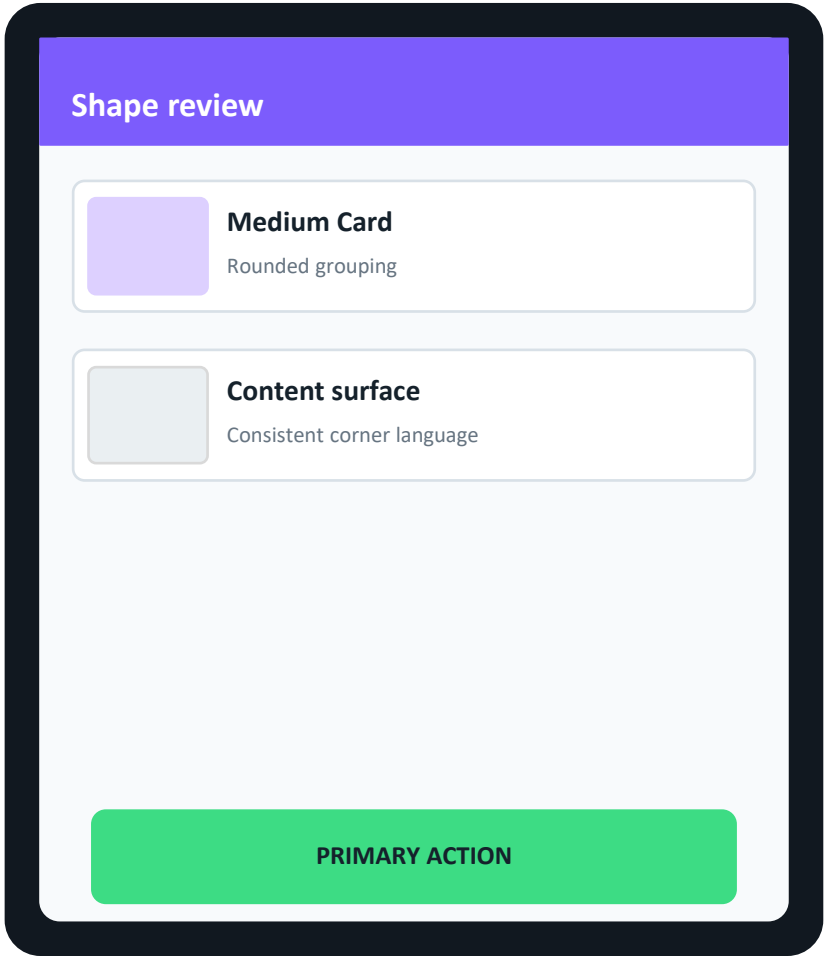
- Small → compact elements
- Medium → cards and containers
- Large → prominent surfaces
- Consistency matters more than memorizing one radius

Small	**4–8dp**	**Compact**
Medium	**12–16dp**	**Cards**
Large	**20–28dp**	**Prominent**

Shape in a Real App

Design review

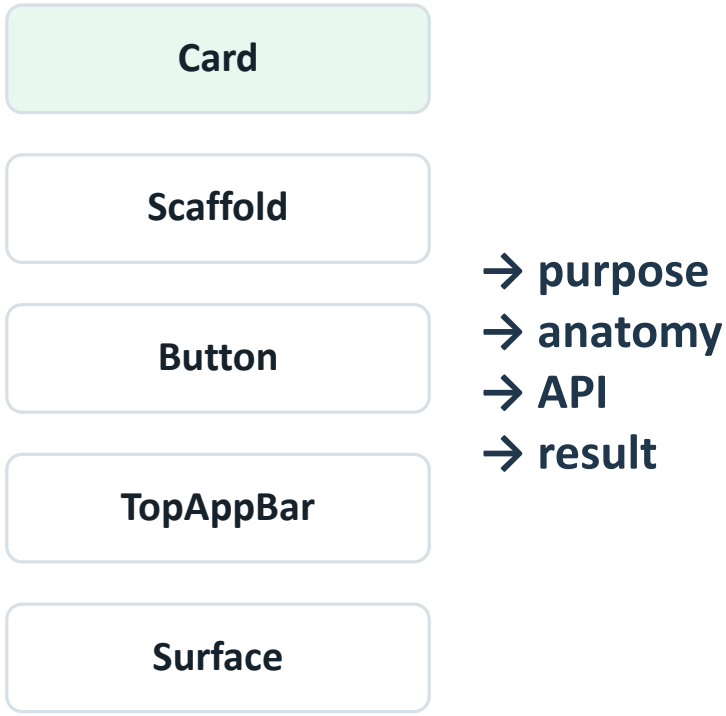
- Button: compact, consistent shape
- Card: medium grouping shape
- Surface: context-dependent
- Do not mix arbitrary radii without reason



Component Thinking

What job needs to be done?

- Group related content → Card
- Structure a screen → Scaffold
- Trigger a primary action → Button
- Represent screen context → TopAppBar
- Create a themed container → Surface



Component Selection Matrix

Requirement → component

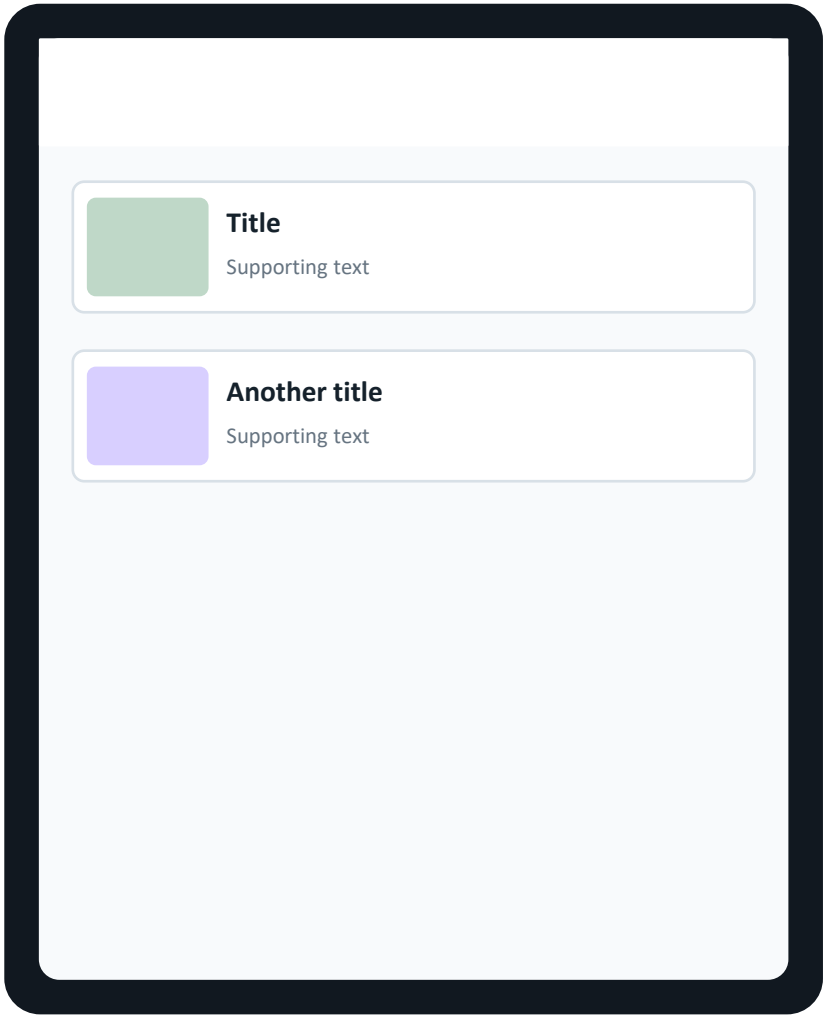
- Group content → Card
- Screen slots → Scaffold
- Main action → Button
- Top-level context → TopAppBar
- Themed container → Surface

Group related content	Card
Screen structure	Scaffold
Primary action	Button
Screen context	TopAppBar
Themed container	Surface

Card Anatomy

An intentional Card contains

- Container
- Image or visual
- Title
- Supporting content
- Spacing
- Interaction if needed

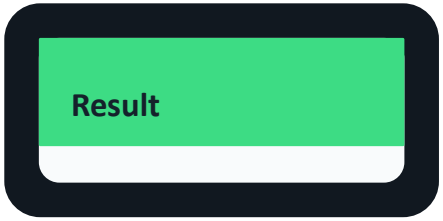


Card Code + Result

Short code is enough

- Use Material 3 Card
- Apply theme shape
- Use Modifier for layout
- Keep content composable

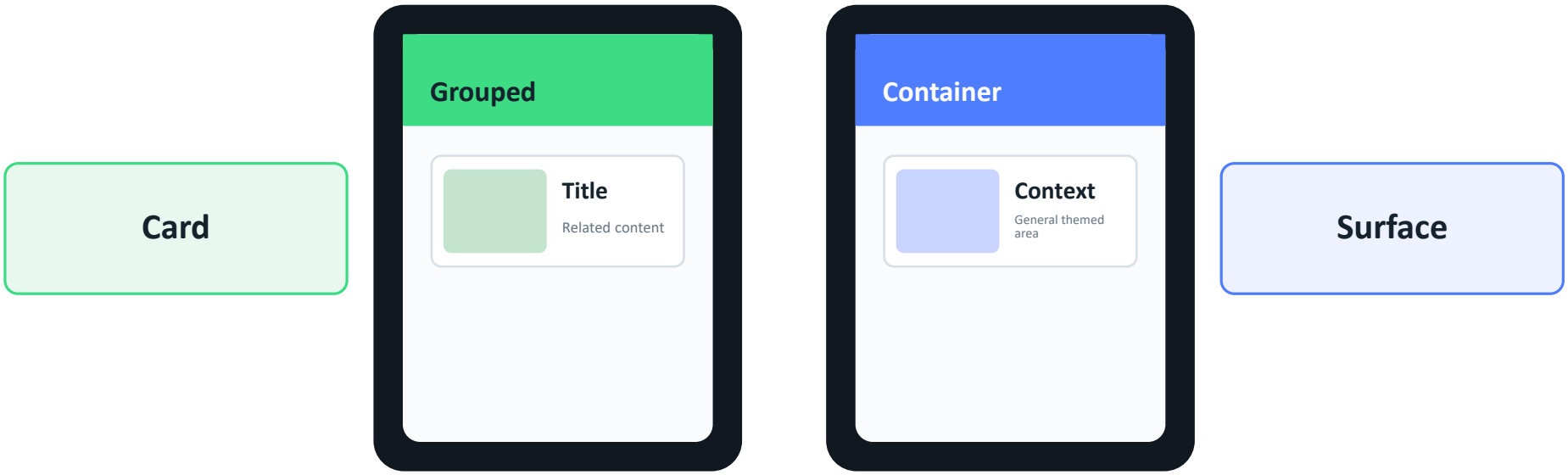
```
Card(
    shape = MaterialTheme.shapes.medium
) {
    Column {
        Text("Hero name")
        Text("Description")
    }
}
```



Surface vs Card

Ask what the container is doing

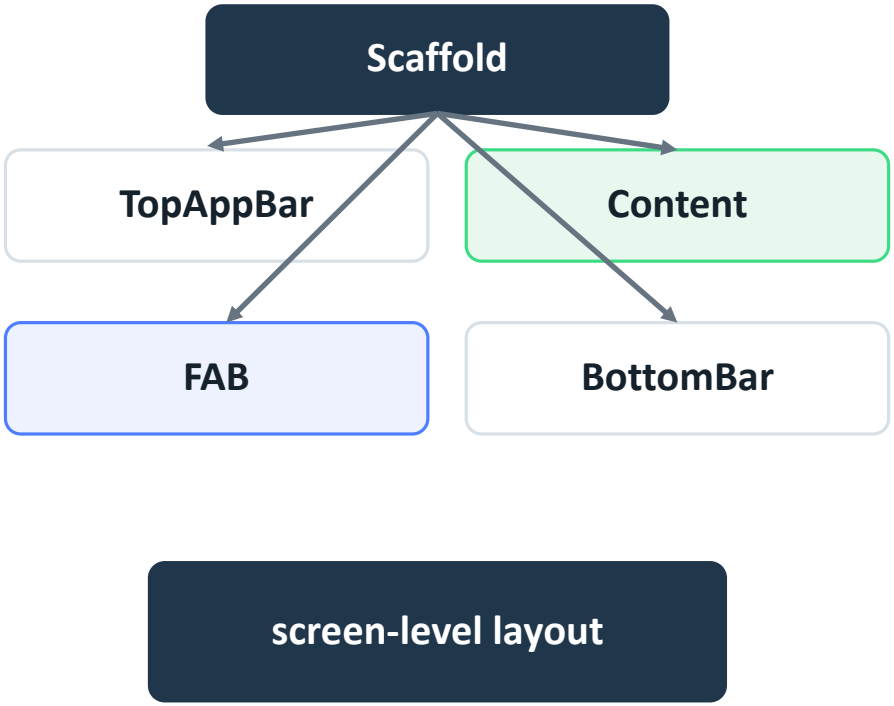
- Card communicates grouped content
- Surface is a more general themed container
- Use the component that matches the semantic job
- Do not use a Card just because it looks attractive



Scaffold Anatomy

Scaffold provides structure

- TopAppBar
- Content
- FloatingActionButton
- BottomBar

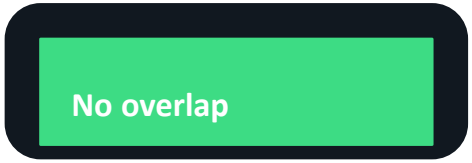


Scaffold + innerPadding

The correct pattern

- Scaffold owns the outer slots
- Content receives innerPadding
- Apply the padding at the content root
- Test on screens with top/bottom bars

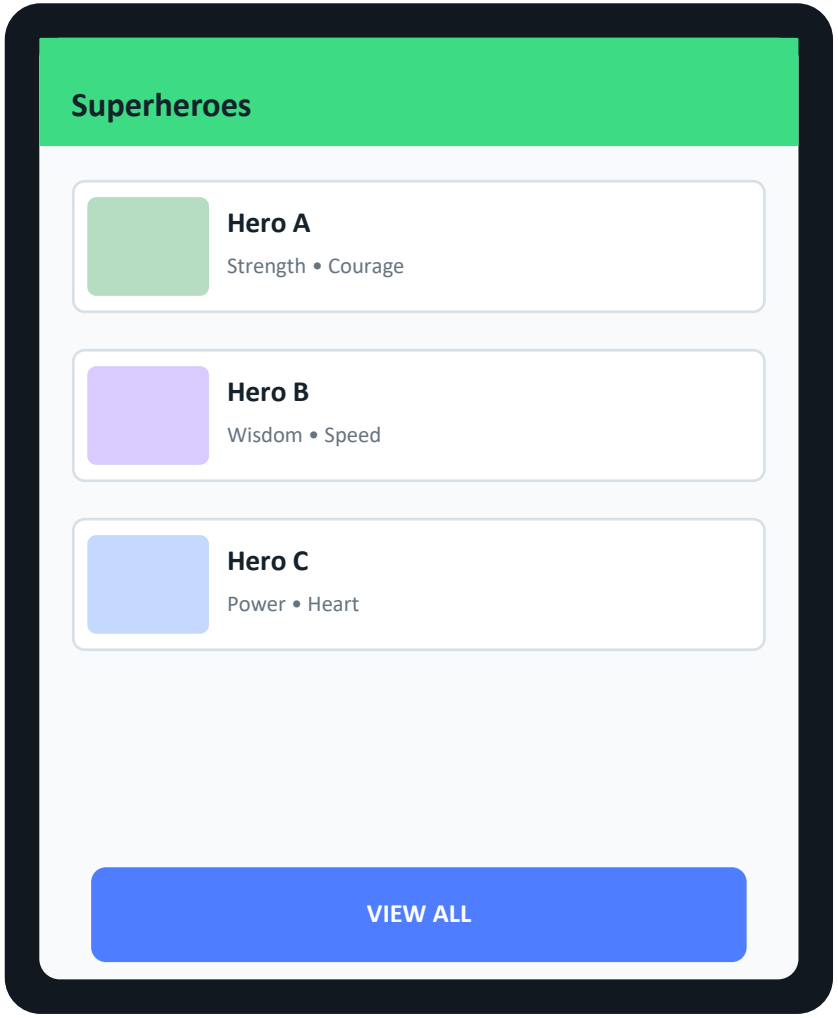
```
Scaffold(
    topBar = { TopAppBar(...) }
) { innerPadding ->
    LazyColumn(
        modifier = Modifier.padding(innerPadding)
    ) {
        ...
    }
}
```



Components in a Complete Screen

One screen, many coordinated decisions

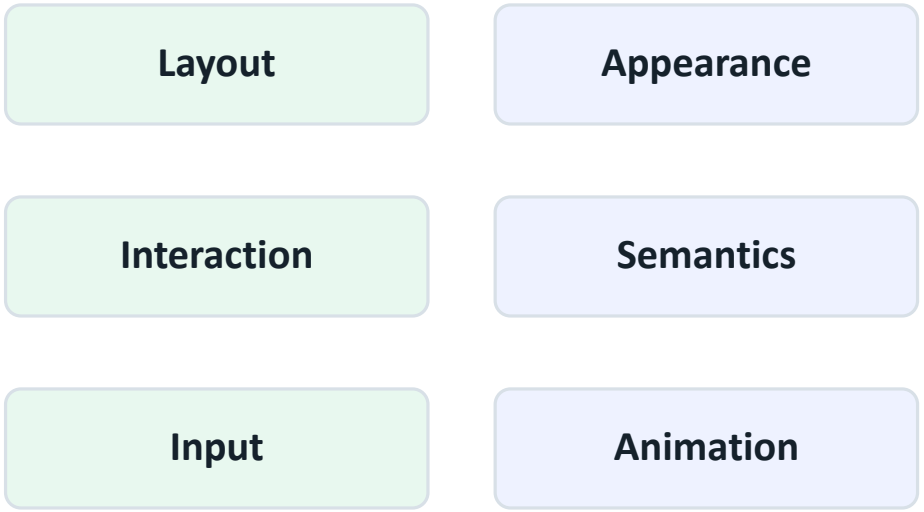
- Scaffold → structure
- TopAppBar → context
- Card → grouping
- Button → action
- MaterialTheme → visual system
- Modifier → layout / interaction



Modifier Mental Model

Modifier describes

- Layout
- Appearance
- Interaction
- Semantics
- Input
- Animation



Modifier Categories

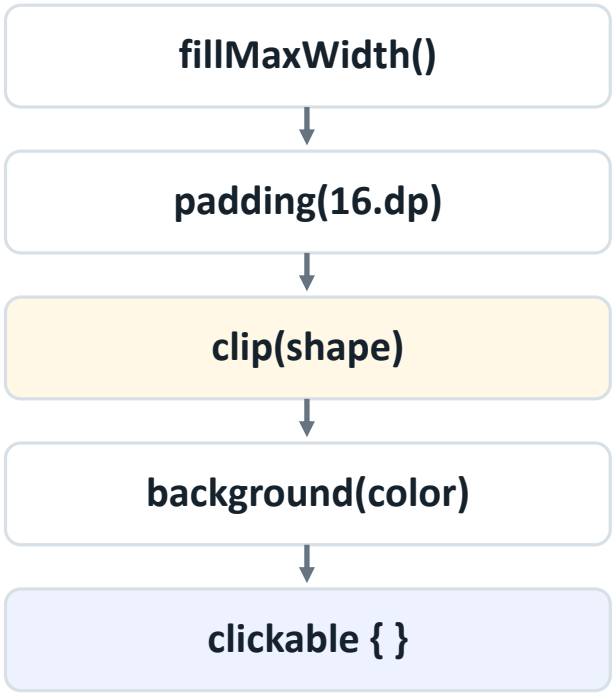
- Layout → padding, fillMaxWidth
- Appearance → clip, background
- Interaction → clickable
- Semantics → semantics
- Animation → animateContentSize

```
Modifier
    .fillMaxWidth()
    .padding(16.dp)
    .clip(shape)
    .background(color)
    .clickable { ... }
```

Modifier Chaining

One chain, multiple responsibilities

- Start with layout bounds
- Add spacing
- Apply shape
- Paint background
- Add interaction



Modifier Order Matters

Compare two chains

- clip → background
- background → clip
- The order changes what is painted/clipped
- Reason about bounds, not memorized rules

```
Modifier
    .clip(shape)

.background(color)
```



```
Modifier

.background(color)
    .clip(shape)
```



Modifier Debugging Lab

For each chain, predict

- What gets padded?
- What gets clipped?
- What gets painted?
- What is clickable?

```
A
.padding()
.background()

B
.background()
.padding()

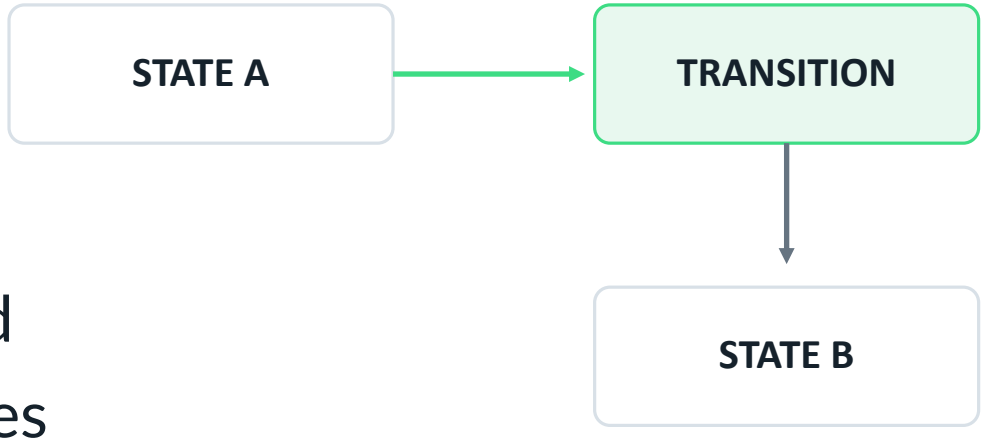
C
.clip()
.background()
.clickable()
```

Predict → run → check

Why Animation?

Useful motion can show

- Something changed
- Something appeared
- Something disappeared
- Something moved or resized
- A relationship between states

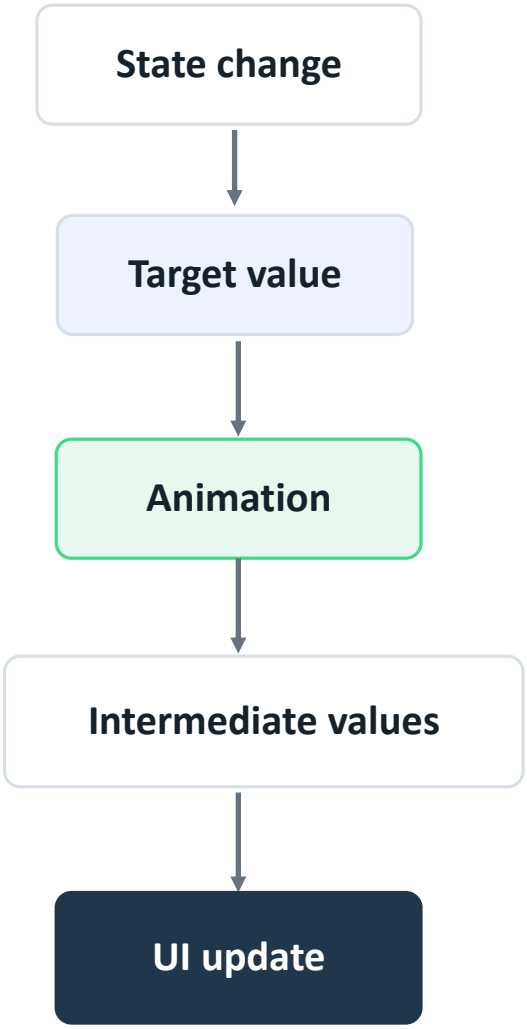


motion explains the change

State → Animation → UI

The flow

- State changes
- Target value changes
- Animation produces intermediate values
- UI reflects the changing value



animate*AsState

Good beginner use case

- One value changes with state
- Color, size, alpha, offset, etc.
- Target value is derived from state
- Compose manages the transition

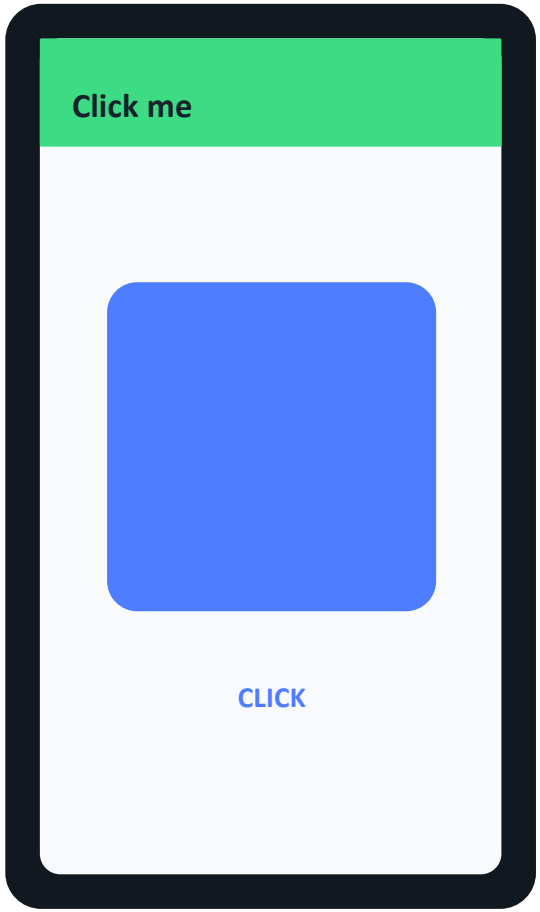
```
val alpha by animateFloatAsState(
    targetValue =
        if (visible) 1f else 0f,
    label = "alpha"
)
```

0.0 → 0.2 → 0.4 → 0.6 → 0.8 → 1.0

State Animation

```
var selected by remember {
    mutableStateOf(false)
}

val size by animateDpAsState(
    targetValue =
        if (selected) 180.dp else 100.dp,
    label = "size"
)
```



AnimatedVisibility

Use when the problem is visibility

- Content exists or does not exist
- Enter and exit can be animated
- State controls visible
- Good for expandable details, messages, secondary content

```
AnimatedVisibility(
    visible = expanded
) {
    Text("More details")
}
```

false → enter → visible

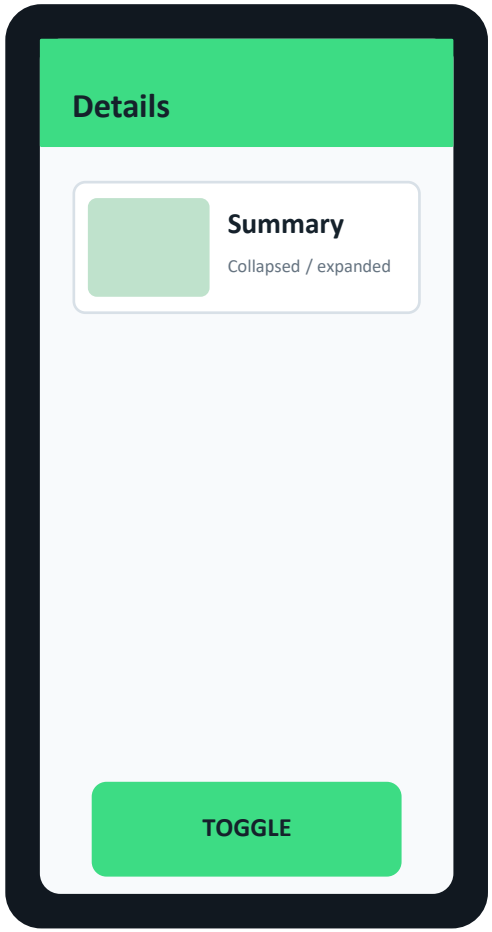
visible → exit → removed

Reveal Details

```
var expanded by remember {
    mutableStateOf(false)
}

Button(
    onClick = { expanded = !expanded }
) {
    Text("Show details")
}

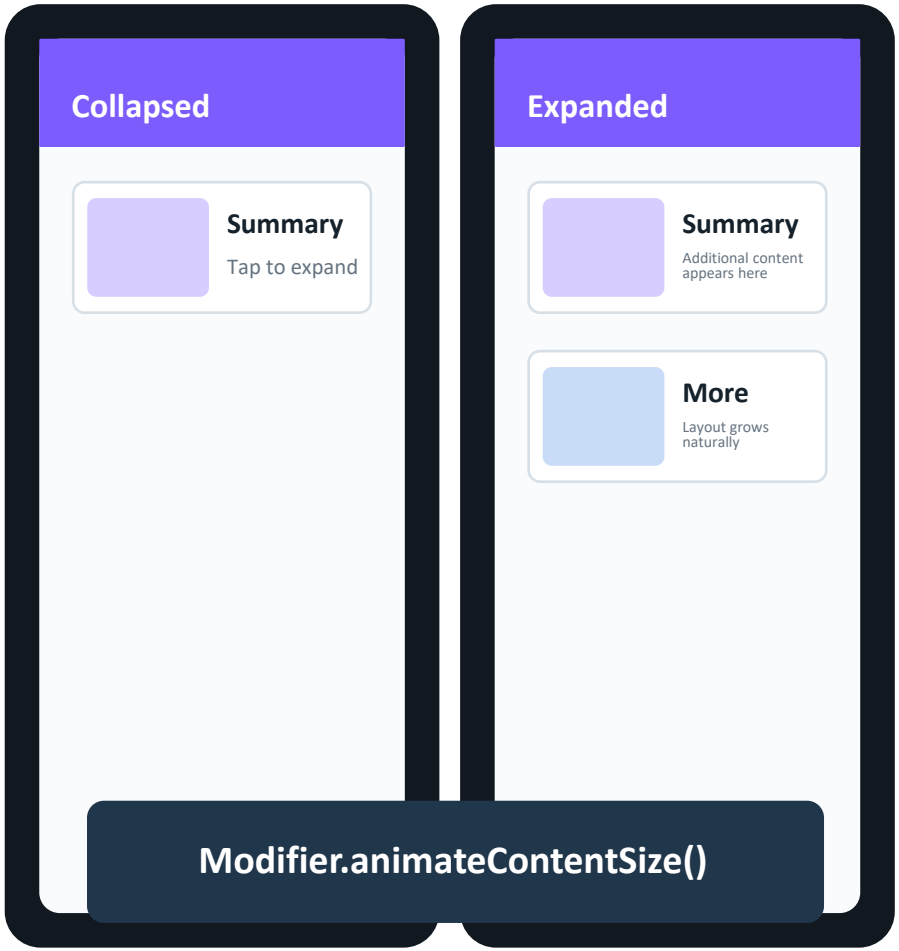
AnimatedVisibility(expanded) {
    Text("More information")
}
```



animateContentSize

A natural use case

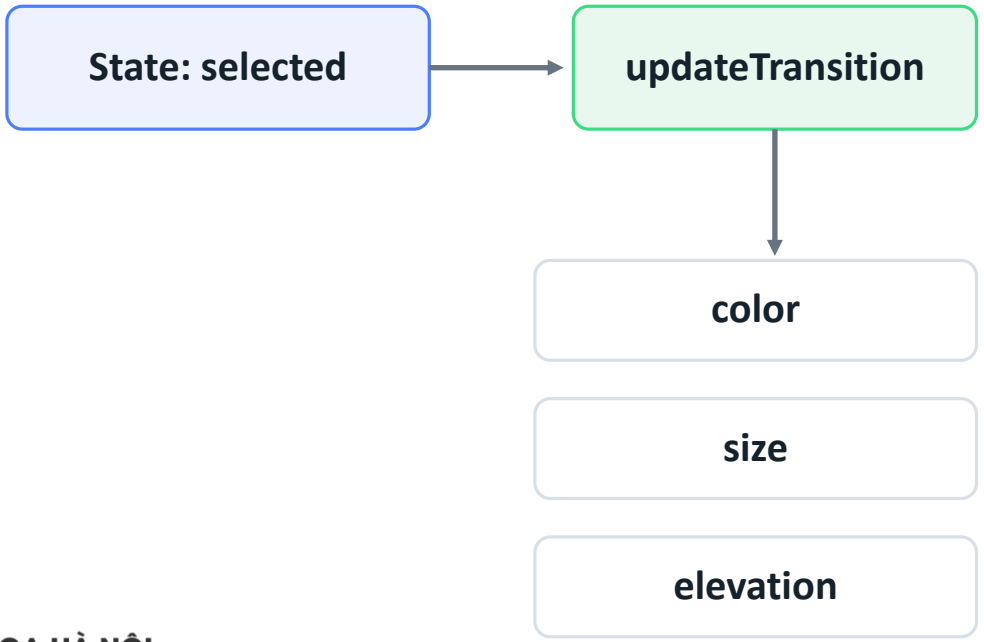
- Expandable text
- Card whose content grows
- Layout-driven size changes
- Avoid manually animating fixed heights for the dynamic content



updateTransition

Use when multiple values move together

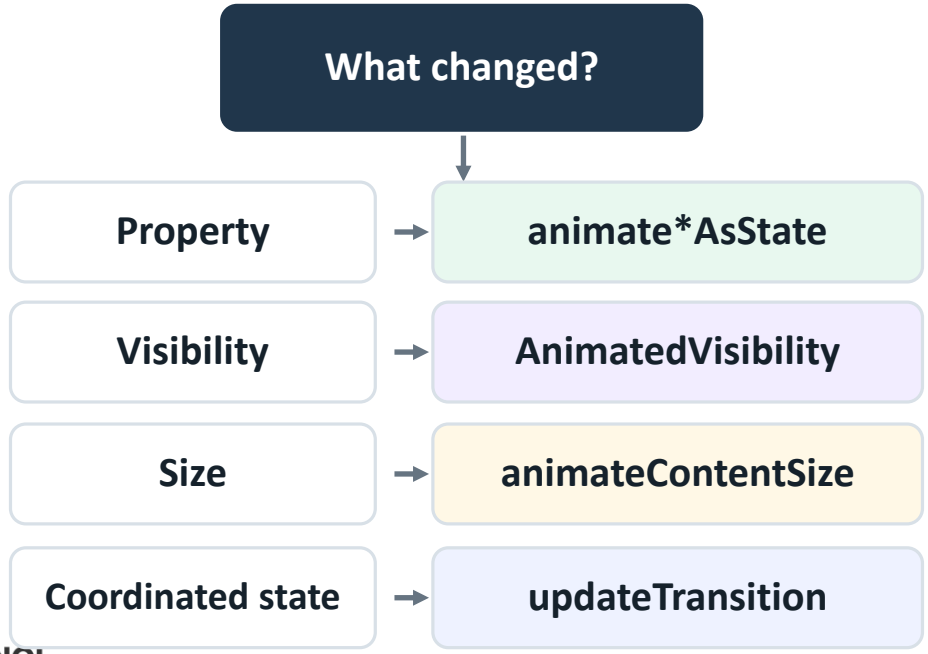
- One state controls several animated properties
- Keeps transitions conceptually coordinated
- Useful for selected/unselected or expanded/collapsed states
- Teach as a next step, not the default



Animation API Decision Tree

Start with the question: what changed?

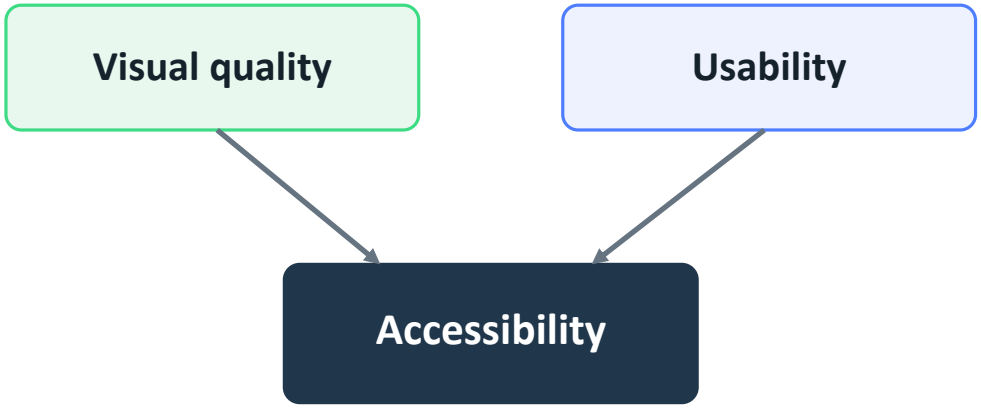
- One property → animate*AsState
- Visibility → AnimatedVisibility
- Content size → animateContentSize
- Multiple coordinated properties → updateTransition



Accessibility Is Product Quality

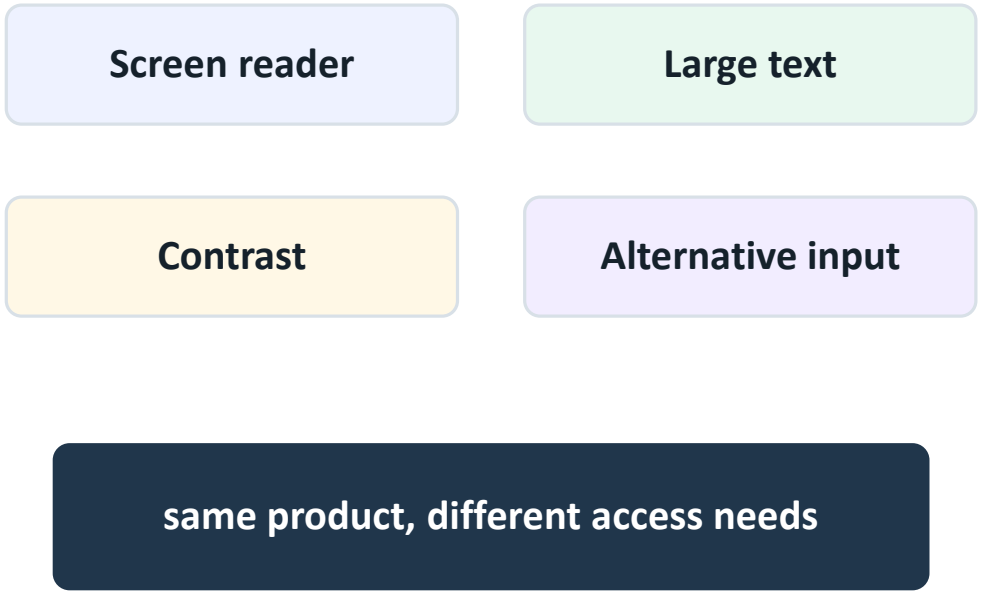
Users may rely on

- Screen readers
- Larger text
- Strong contrast
- Alternative input
- Non-color cues
- Predictable interaction



Who Benefits?

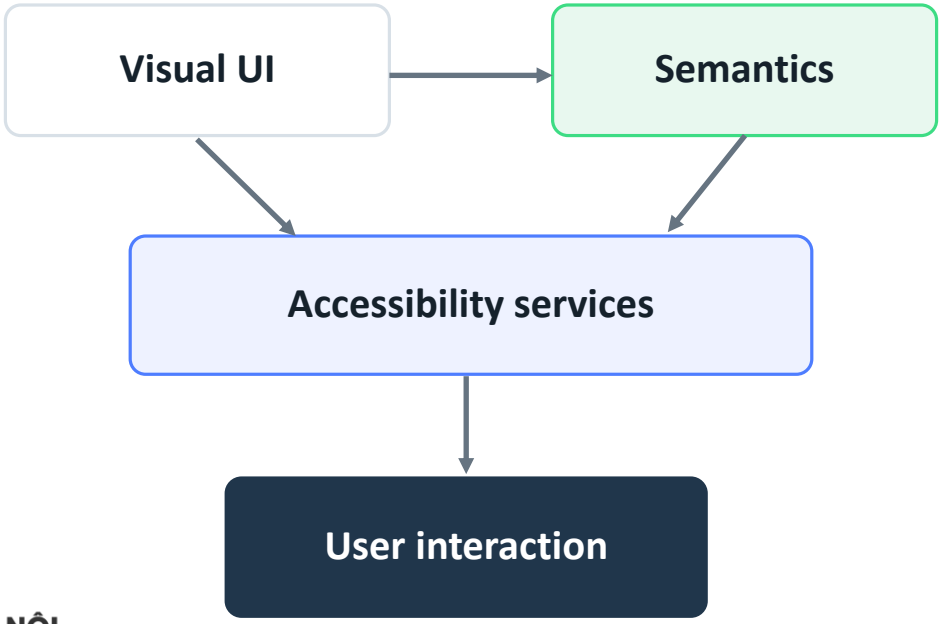
- Screen reader user navigating a list
- Low-vision user increasing text size
- User needing stronger contrast
- User using switch / alternative input
- Anyone affected by unclear interaction



Visual UI + Semantics

The semantic layer

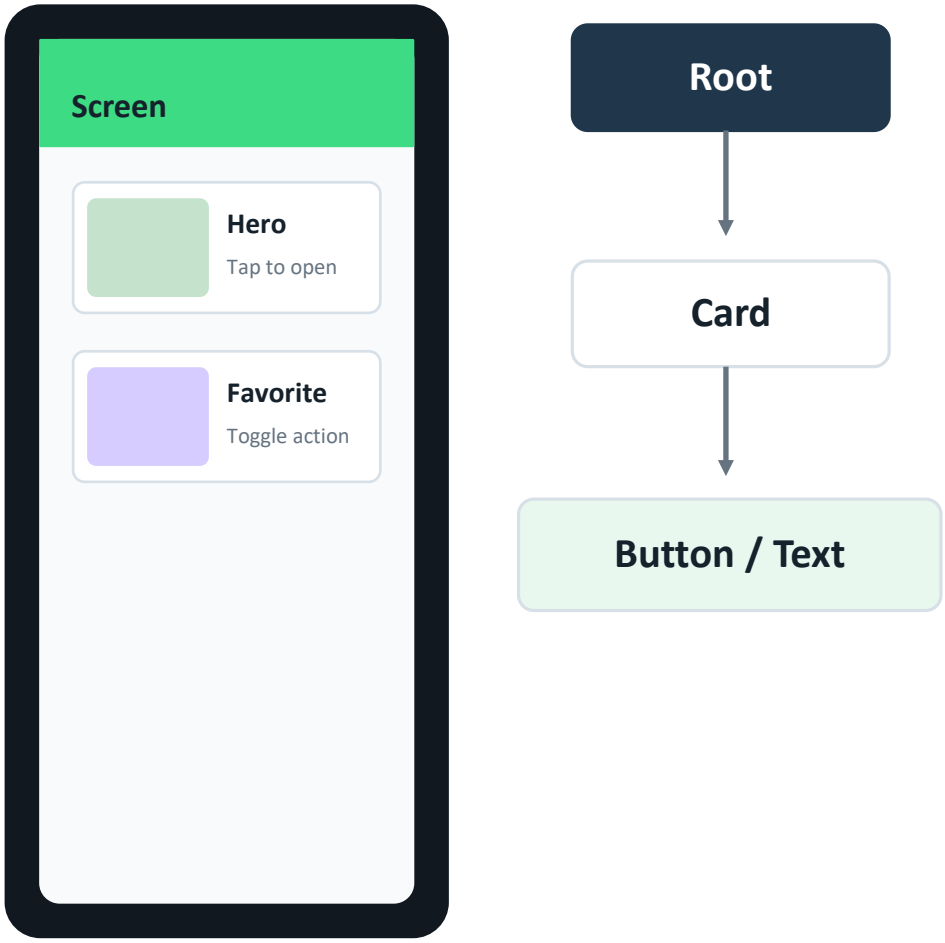
- Visual UI communicates appearance
- Semantics communicate meaning
- Accessibility services consume semantics
- Users interact through those services



What Is the Semantics Tree?

Semantics can expose

- Text / labels
- Roles
- States
- Content descriptions
- Actions
- Testable meaning



Content Descriptions

A correct choice depends on meaning

- Meaningful image → describe it
- Informational image → describe the information
- Decorative image → contentDescription = null
- Redundant imagery → avoid duplicate announcements

```
Image(
    painter = painterResource(...),
    contentDescription =
        "Superhero logo"
)

// decorative
contentDescription = null
```

When contentDescription = null Is Correct

Decision examples

- Meaningful → description
- Decorative → null
- Already described by surrounding control → avoid redundancy
- Image carrying essential information → description

Meaningful	**YES**
Decorative	**NO**
Redundant	**AVOID DUPLICATE**
Informational	**YES**

Semantics, Role, and State

Useful tools

- contentDescription
- role
- stateDescription
- clearAndSetSemantics
- Semantics modifiers for custom controls

```
Modifier.semantics {
    contentDescription =
        "Add to favorites"
    // role / state can be added
    // when they clarify meaning
}
```

Touch Targets + Text Scaling

Audit the UI for

- Adequate interactive target size
- No cramped icon-only controls
- Large text scaling
- Flexible layout
- Readable content at different sizes



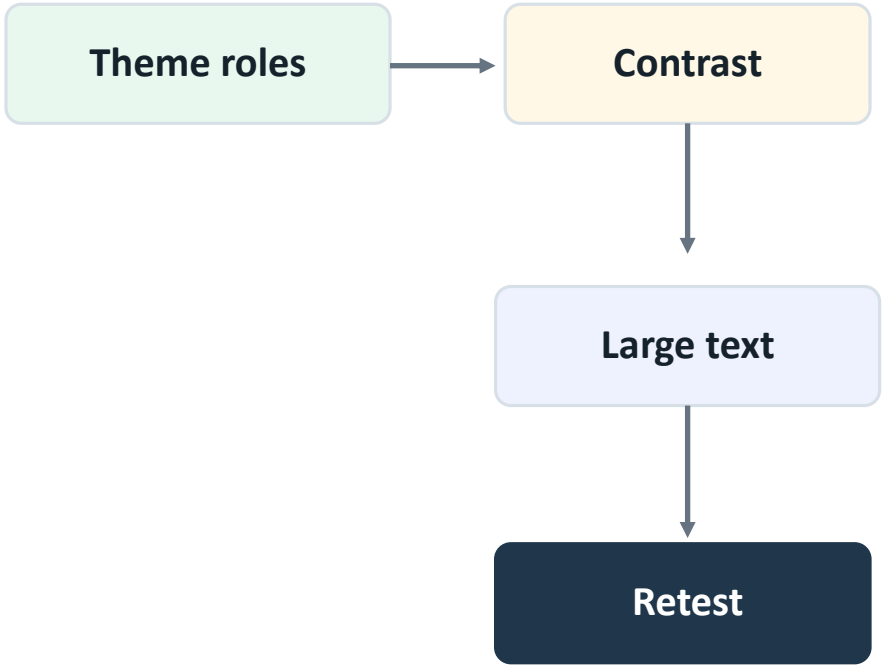
48dp
minimum target
mental model

Then test with large text

Contrast + Accessibility

The design loop

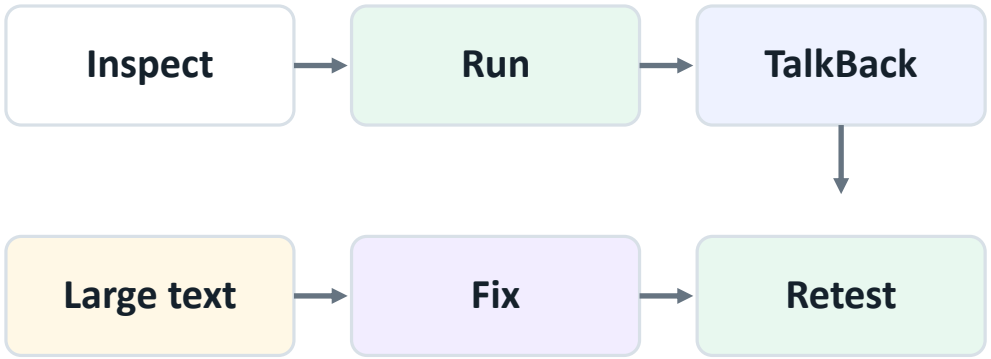
- Choose semantic color roles
- Render in light and dark themes
- Check contrast
- Test large text
- Adjust and retest



Accessibility Testing Workflow

Manual review workflow

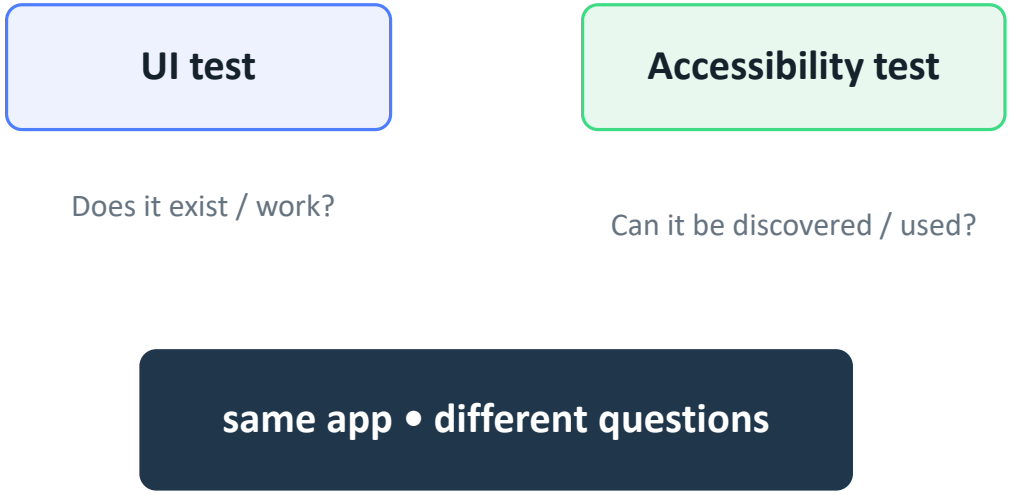
- Inspect semantics
- Run the app
- Use TalkBack
- Increase text size
- Check contrast / targets
- Fix → retest



UI Testing vs Accessibility Testing

Testing asks different questions

- UI test → does the behavior/state work?
- Accessibility test → can the user discover and understand it?
- Semantics can support both
- Manual testing still matters



Compose Semantics Test

A minimal assertion

- Find by visible text
- Find by content description when appropriate
- Assert existence or state
- Use semantics as a testable contract

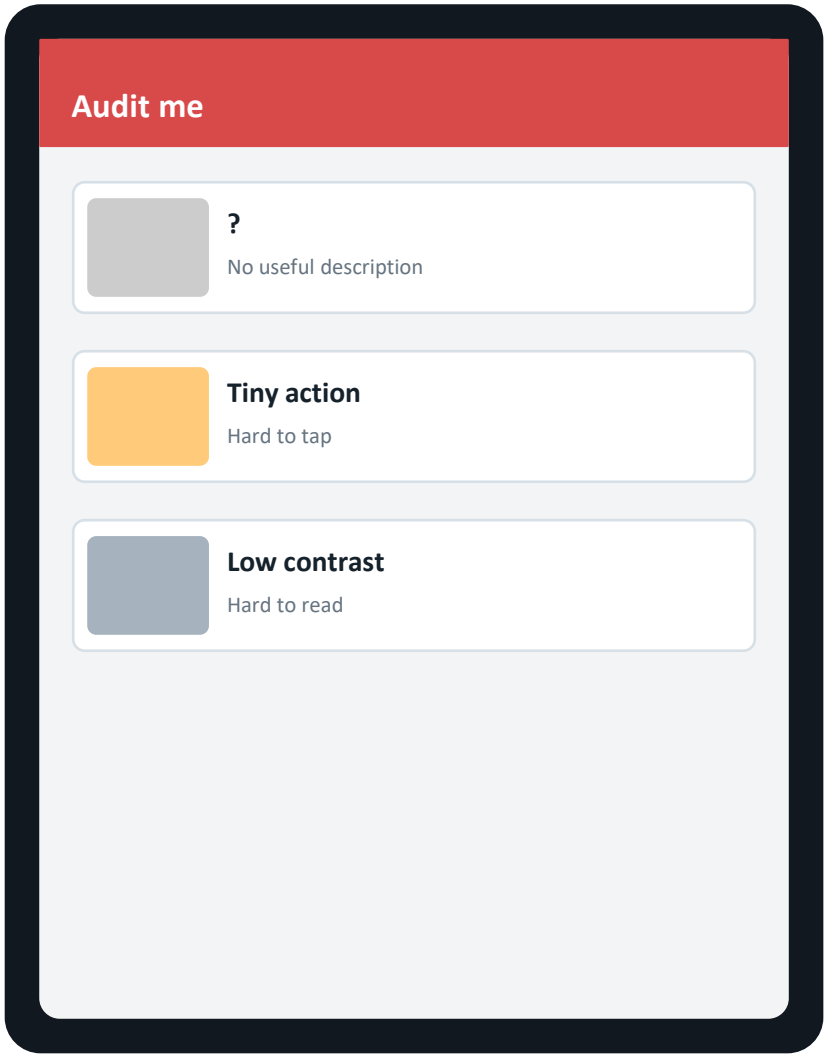
```
composeTestRule
    .onNodeWithText("Hello")
    .assertExists()

// other examples
onNodeWithContentDescription(...)
onNode(...)
```

Accessibility Audit Lab

Find at least five issues

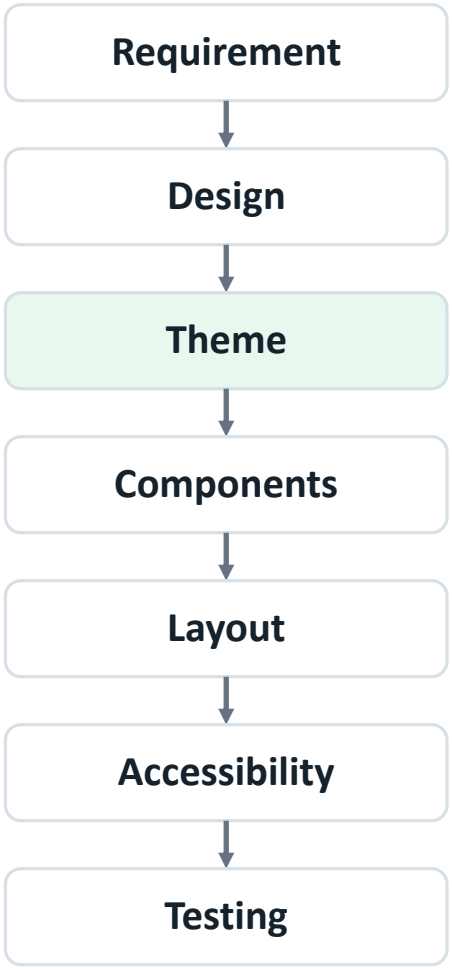
- Missing description
- Poor contrast
- Tiny interaction target
- Unclear state
- Redundant announcement
- Large text overflow



Superheroes: Requirements → Design

Design flow

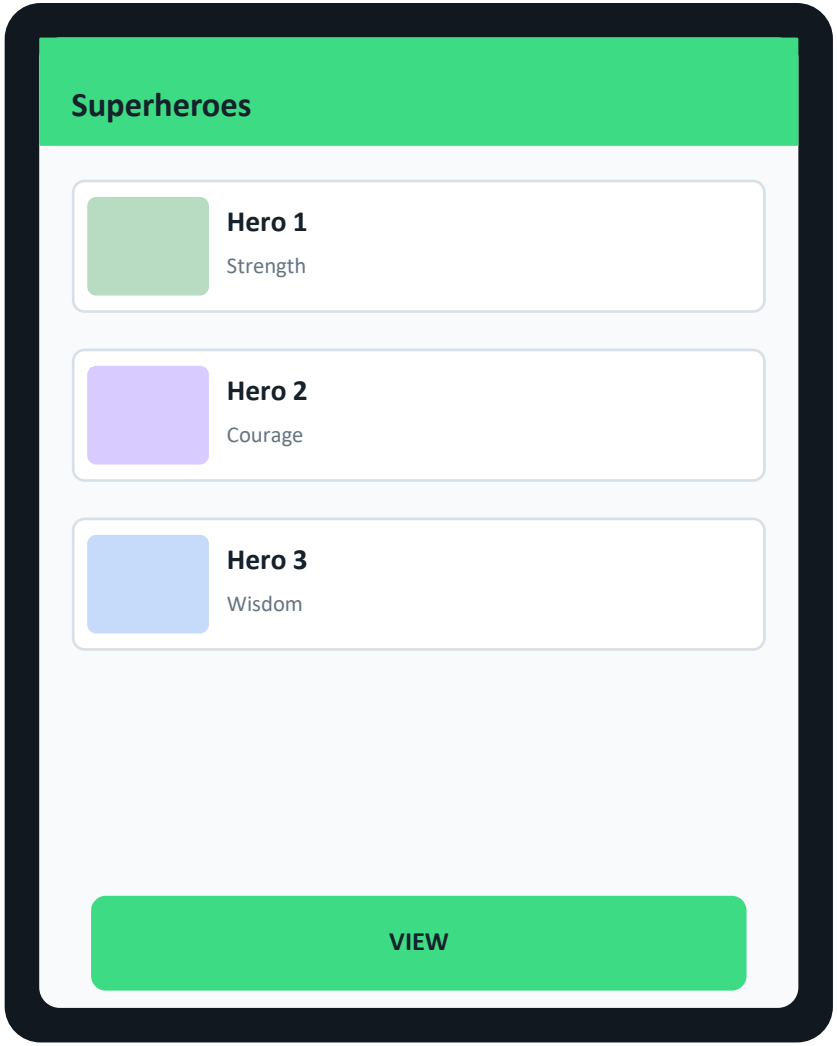
- Requirement
- Design
- Theme
- Components
- Layout
- Accessibility
- Testing



Superheroes UI Anatomy

Look for

- TopAppBar
- Card
- Image
- Typography
- Shape
- Modifier
- Spacing
- Theme colors



Superheroes: Code → UI Mapping

Read the screen in layers

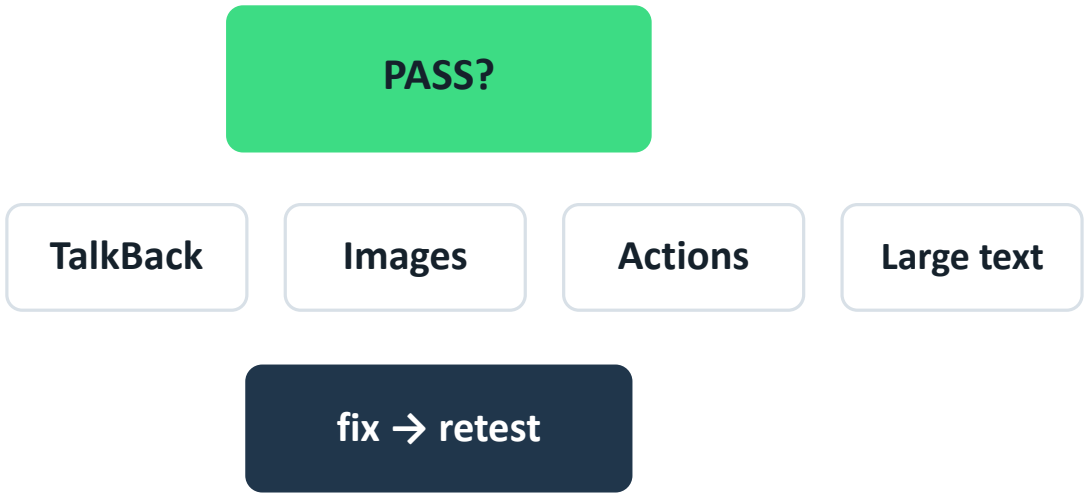
- Scaffold → screen structure
- LazyColumn → list
- Card → grouped item
- Text/Image → content
- MaterialTheme → system
- Modifier → layout

```
Scaffold(...) { innerPadding ->
    LazyColumn(
        contentPadding = innerPadding
    ) {
        items(heroes) { hero ->
            Card(...) { ... }
        }
    }
}
```

Superheroes Accessibility Review

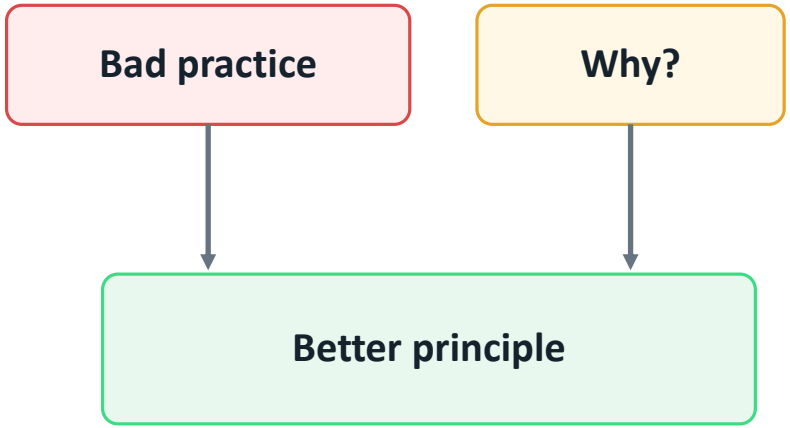
Audit questions

- Can the list be navigated?
- Are meaningful images described?
- Are actions labeled?
- Does text remain readable?
- Do touch targets remain usable?



Common Mistakes

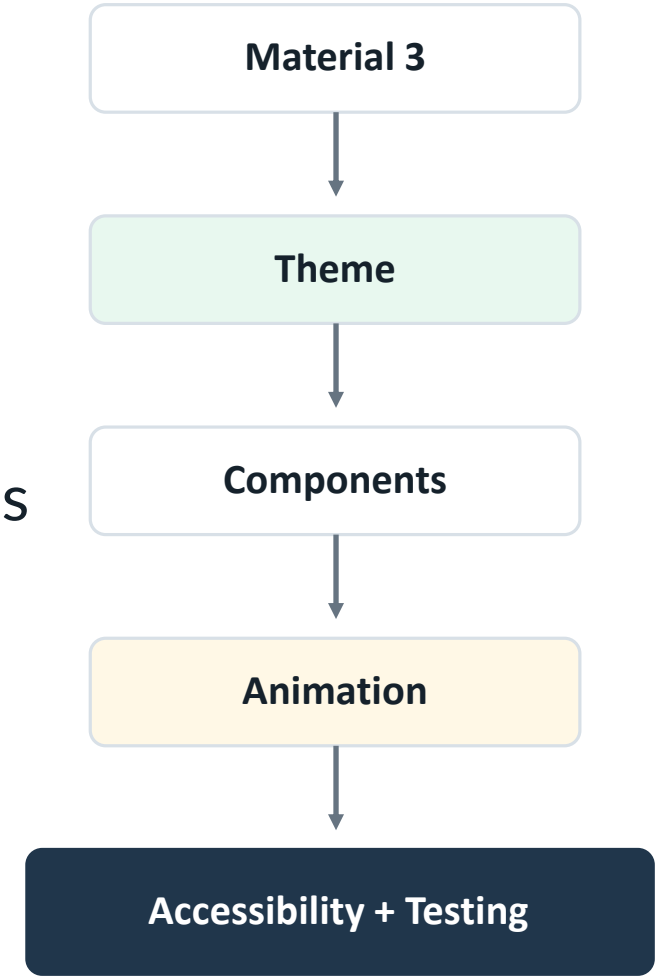
- Hard-coded colors / typography
- No use of MaterialTheme
- Wrong Modifier order
- Arbitrary padding / shape
- Excessive animation
- Missing semantics
- Contrast problems
- No dark-theme test
- No accessibility test
- Poor hierarchy
- Wrong component for the job
- Monolithic UI blocks



Summary

The production-quality loop

- Material Design → Material 3
- Theme → Color / Typography / Shape
- Components → Layout
- Animation → meaningful state change
- Accessibility → semantics + usable visuals
- Testing → validate the result



HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !